

UNIT - IV

4

JSP and Web Services

Syllabus

JSP : Introduction to Java Server Pages, JSP and Servlets, running JSP applications, Basic JSP, JavaBeans classes and JSP, Support for the Model-View-Controller paradigm, JSP related technologies. **Web Services** : Web Service concepts, Writing a Java Web Service, Writing a Java web service client, Describing Web Services : WSDL, Communicating Object data: SOAP. **Struts** : Overview, architecture, configuration, actions, interceptors, result types, validations, localization, exception handling, annotations.

Contents

4.1	Introduction to Java Server Pages	
4.2	JSP and Servlets	April-18, Marks 5
4.3	Basics of JSP	April-18, May-18, 19, Dec.-18, 19, Marks 5
4.4	Implicit Objects	May-18, 19, Dec.-18, Marks 5
4.5	JavaBeans Classes and JSP	May-18, Marks 5
4.6	Support for the Model-View-Controller Paradigm	Dec.-18, Marks 6
4.7	Web Services Concepts	May-18,19, Dec.-18, Marks 8
4.8	Writing a Java Web Service	
4.9	Describing Web Services : WSDL	
4.10	Communicating Object Data : SOAP	
4.11	Overview	Dec.-18, Marks 4
4.12	Architecture	May-18, 19 Marks 6
4.13	Execution of First Struts Applications	
4.14	Configuration	Dec.-18,19, Marks 6
4.15	Actions	Dec.-18, May-19, Marks 6

4.16	Interceptors		May-18,		Marks 6
4.17	Result Types				
4.18	Validations				
4.19	Localization				
4.20	Exception Handling				
4.21	Annotations				
4.22	Multiple Choice Questions				

Part I : JSP

4.1 Introduction to Java Server Pages

- JSP stands for Java Server Pages.
- It is an alternative way than the servlet to build the dynamic web pages. It is built on top of the servlet.

4.2 JSP and Servlets

SPPU : April-18, Marks 5

- Following are some advantages of Java Server pages -
 1. JSP is useful for **server side programming**.
 2. JSP can be used **along with servlets**. Hence **business logic** for any application can be developed using JSP.
 3. **Dynamic contents** can be handled using JSP because JSP allows scripting and element based programming.
 4. JSP allows creating and using our own **custom tag libraries**. Hence any application specific requirements can be satisfied using custom tag libraries. This helps the developer to develop any kind of application.
 5. JSP is a **specification and not a product**. Hence developers can develop **variety of applications** and add up to performance and quality of software products. Due to this many companies are ready to invest in JSP technology.
 6. JSP is an essential component of J2EE. Hence using JSP it is possible to develop simple as well as complex applications.

Difference between Servlet and JSP

Sr. No.	JSP	Servlet
1.	JSP is a scripting language that generates dynamic content.	Servlets are the Java programs that can be compiled to generate dynamic content.
2.	JSP runs slower than servlet.	Servlet runs faster than JSP.
3.	In Model View Controller JSP acts as view.	In Model View Controller servlet acts as controller.
4.	JSP can build custom tags.	There is no facility of creating custom tags.
5.	JSP can directly call Java beans.	Servlets have no facility of calling Java bean.
6.	It is easier to code in JSP.	The servlets are basically complex Java programs.

Review Question

1. Write advantages of JSP over servlets. Also explain the life cycle of JSP.

SPPU : April-18, Marks 5**4.3 Basics of JSP****SPPU : April-18, May-18, 19, Dec.-18, 19, Marks 5****4.3.1 JSP Document Structure**

Every JSP document contains four elements -

1. **Template text** : The XML of HTML code can be directly used in JSP document. This is called the static or fixed part of the document. This markup is called template text. This part is not modified by JSP container.
2. **Action elements** : The action elements are used to create the documents dynamically. The action elements is written in the form
 - i) Opening tag
 - ii) Action body
 - iii) Closing tag.

For example

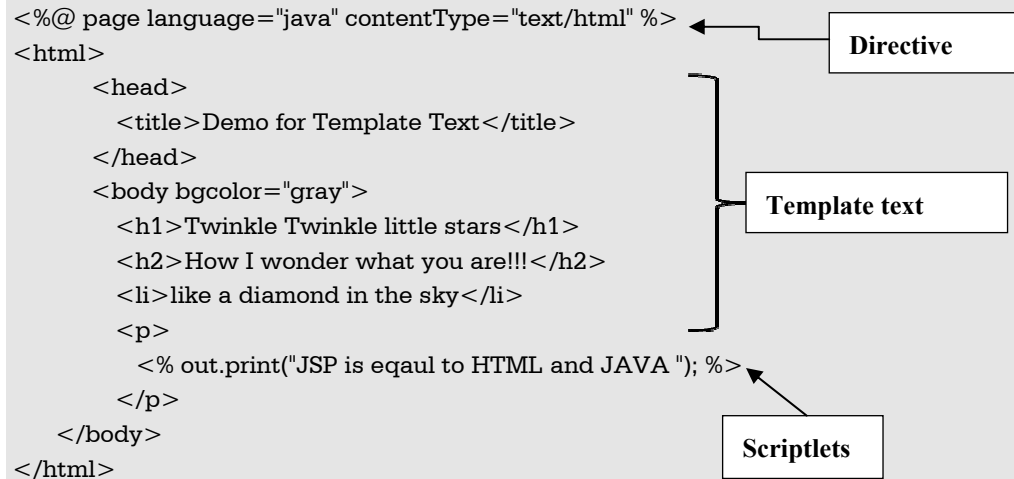
<code><jsp:include></code>	This tag works as a subroutine. It includes the response from servlet or JSP when request is being processed.
<code><jsp:forward></code>	This tag helps in forwarding the current request to servlet or to JSP page.
<code><jsp:element></code>	This tag generated the XML elements dynamically.
<code><jsp:text></code>	This tag is used to handle template text. When JSP pages are written as XML documents then this tag is used.

3. **Directives** : JSP directives control the processing of entire JSP Page. It gives direction to the server regarding the processing of a page. The directive elements are enclosed within the `<%@` `%>` delimiters. For example -

```
<%@page import="java.io.*" %>
<%@page language="java" %>
<%@page contentType="text/html" %>
```

4. **Scriptlets** : Scriptlets are nothing but java code enclosed within `<%` and `%>` tags. For example we can write following statement in JSP document

```
<p>
  <% out.print("JSP is equal to HTML and JAVA "); %> //Scriptlet
</p>
```

Example of JSP document**4.3.2 JSP Processing**

- JSP pages can be processed using **JSP container** only. Following are the steps that need to be followed while processing the request for JSP page -
 - Client makes a request for required JSP page to the server. The server must have JSP container so that JSP request can be processed. For instance : Let the client makes a request for *xyz.jsp* page.
 - On receiving this request the JSP container searches and then reads the desired JSP page. Then this JSP page is straightaway converted to corresponding **servlet**. Basically any JSP page is a combination of template text and JSP element. Every **template text** is translated into corresponding **println** statement.

For instance :

**Fig. 4.3.1**

Every **JSP element** is converted into corresponding **Java code**. This phase is called **translation phase**. The output of translation phase is a servlet. For instance : our *xyz.jsp* gets converted to *xyzServlet.java*.

- This servlet is then compiled to generate the servlet class file. Using this class the response can be generated. This phase is called **request processing phase**.
- The JSP container thus **executes** the servlet class file.
- A requested page is then returned to the client as a **response**. Refer Fig. 4.3.2.

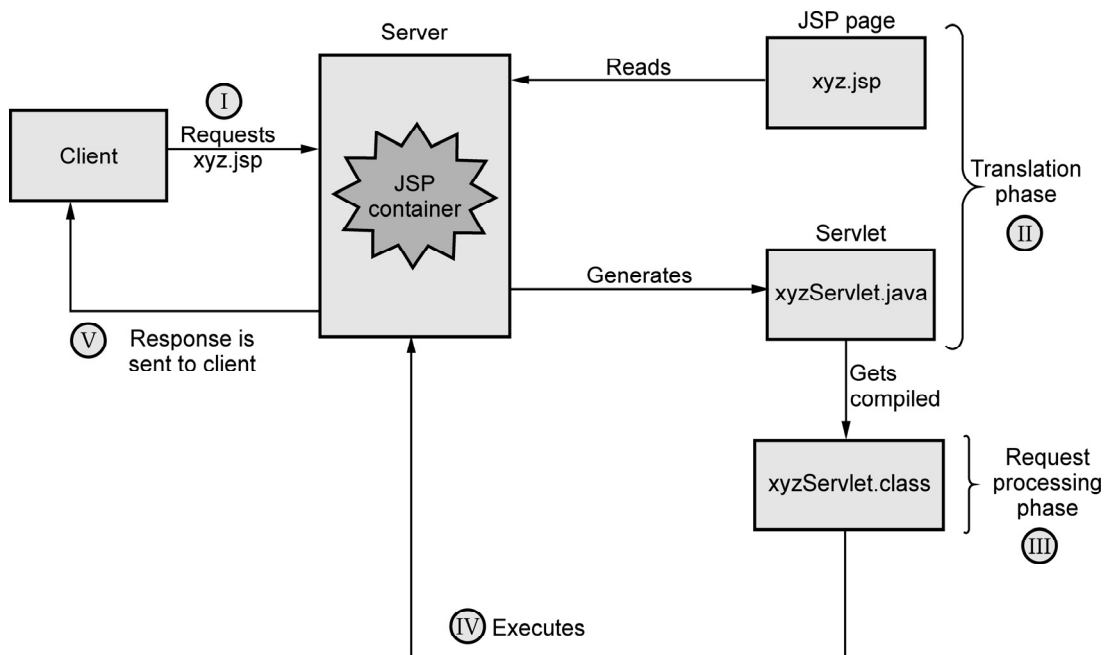


Fig. 4.3.2 JSP processing

4.3.3 Running JSP Applications

For execution of JSP pages following installations are needed.

1. **Java Development Kit** : Download and install the latest version of JDK.
2. **Tomcat WebServer** : The Tomcat is a web container provided by Apache Web server. Now a days XAMPP software package is installed for running JSP, Servlets and PHP programs.

Note that before running any JSP page it is essential to start the tomcat by double clicking the startup.bat file present in the tomcat directory.

Example 4.3.1 Write a simple JSP page for displaying the message "This is my first JSP page!!!"

SPPU : May-18, Marks 5

Solution : We follow these steps to get the JSP page displayed -

Step 1 : First of all open some text editor like Notepad and type the following code.

hello.jsp

```

<%@ page language="java" contentType="text/html" %>
<%@ page import = "java.util.*" %>
<html>
<!-- This is basic JSP page -->
<title>JSP Demo </title>

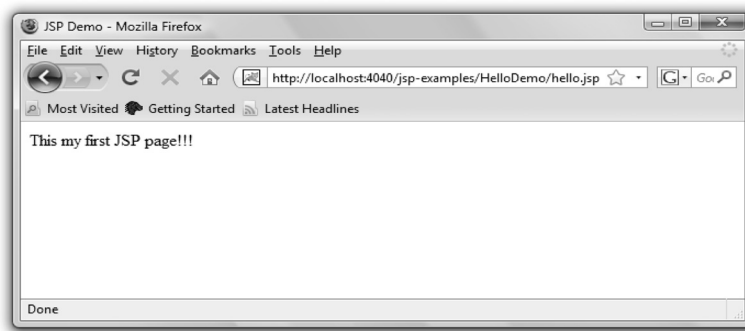
```

```
<body>
<!-- Displaying the message on the browser --%>
    <% out.println("This my first JSP page!!!"); %>
</body>
</html>
```

Step 2 : Copy this file to webapps directory of your tomcat folder.

Step 3 : Start the Tomcat Web server.

Step 4 : Open some suitable web browser like Firefox or Internet Explorer. Type the path for the JSP document and the output will be as follows -



Script explanation

The first line is the **page directive** statement

```
<%@ page language="java" contentType="text/html" %>
```

using this line we are specifying that the language used in this script is Java and the contentType specifies the MIME type which is text/html.

Then on the next line we have imported the Java library package **java.util** using import statement -

```
<%@ page import ="java.util.*" %>
```

We can specify the JSP comments in two ways

```
<!--HTML style comment statement -->
```

```
<%-- JSP comment statement --%>
```

As JSP is commonly used along with HTML, it also supports the HTML comments.

For displaying the message on the web browser we have written the message using the Java statement

```
out.println("This my first JSP page!!!");
```

Normally any Java code must be written within `<%` and `%>`.

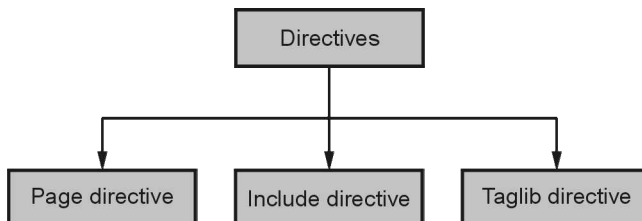
Then all the corresponding tags are closed.

4.3.4 Directives

JSP directives control the processing of entire JSP Page. It gives direction to the server regarding the processing of a page.

Syntax

```
<%@ directive name [attribute name="value" attribute name="value" .....]%>
```



1. Page directive :

The page directive is used to provide the information about the page. Various attributes of page directive are -

- | | | |
|----------------|--------------------|------------------|
| i) import | ii) language | iii) ContentType |
| iv) extends | v) session | vi) buffer |
| vii) autoFlush | viii) isThreadSafe | ix) info |
| x) errorPage | xi) IsErrorPage | |

For example -

```
<%@page import="java.io.*" %>
<%@page language="java" %>
<%@page contentType="text/html" %>
```

2. Include directive :

Include directive is used to copy the content of one JSP page to another. It's like including the code of one file into another.

For example

```
<% @include file=calculate.jsp %>
```

3. Taglib directive :

This directive basically allows user to use custom tags in JSP. The custom tags are those which are created by user.

For example -

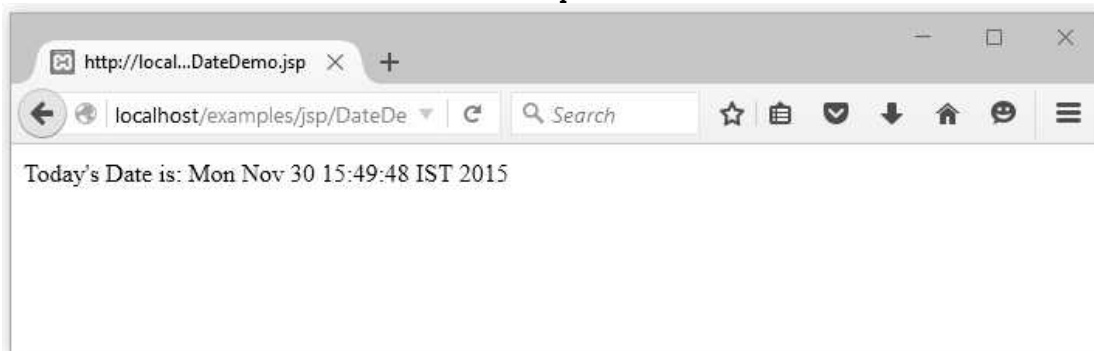
```
<%@ taglib uri="mytaglib" prefix="sampletag" %>
```

Example 4.3.2 Write a JSP program to display current date and time.

Solution : DateDemo.jsp

```
<%@ page language="java" contentType="text/html" %>
<%@ page import = "java.util.*" %>
<html>
<body>Today's Date is:
    <%= new Date().toString() %>
</body>
</html>
```

Output



4.3.5 Comments

We can specify the JSP comments in two ways

```
<!--HTML style comment statement -->
<%-- JSP comment statement --%>
```

As JSP is commonly used along with HTML, it also supports the HTML comments.

4.3.6 Scripting Elements

There three types of scripting elements in JSP.

1. Expression
2. Scriptlet
3. Declaration

1. Expression

The expression tag is used to represent the expression in JSP page. The syntax of writing expression is -

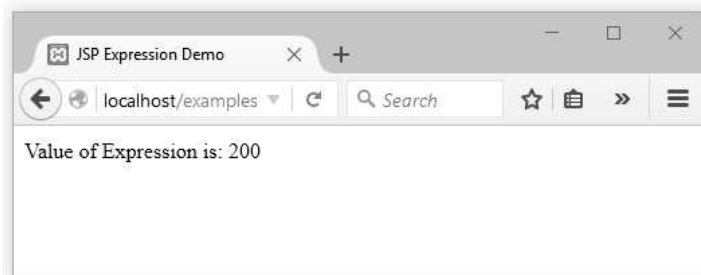
```
<%= Java Expression %>
```

For example -

```
<html>
<head>
    <title>JSP Expression Demo</title>
</head>
```

```
<body>
  Value of Expression is :
  <%= (10*20) %>
</body>
</html>
```

Output



2. Scriptlet

- The code that appears between the `<%` and `%>` delimiters is called *a scriptlet*. Scriptlets are nothing but java code **enclosed within** `<%` and `%>` tags.
- The everything other than a JSP statement in the JSP is called template text. For example.

Templatetext.jsp

```
<%@ page language="java" contentType="text/html" %>
<html>
  <head>
    <title>Demo for Tempplate Text</title>
  </head>
  <body bgcolor="gray">
    <h1>Twinkle Twinkle little stars</h1>
    <h2>How I wonder what you are!!!</h2>
    <li>like a diamond in the sky</li>
    <p>
      <% out.print("JSP is eqaul to HTML and JAVA "); %>
    </p>
  </body>
</html>
```

In the above code the boldfaced code is a template text. The output for the above code will be

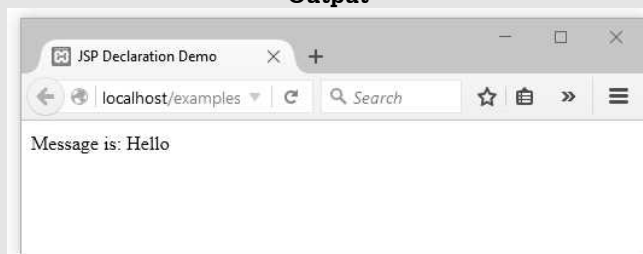
Output**3. Declaration**

We can declare a variable or method in JSP inside declaration tag. We can declare static member, instance variable and methods inside declaration tag. Syntax of declaration tag -

```
<%! Declaration code %>
```

For example -

```
<html>
<head>
  <title>JSP Declaration Demo</title>
</head>
<%!
  String msg = "Hello";
%>
<body>
  Message is:
  <% out.println(msg); %>
</body>
</html>
```

Output

The above JSP code contains the declaration within `<%! %>` tag.

Example 4.3.3 *Develop a simple JSP application that accepts user's age and displays a message according to the following rules :*

- i) Age < 15 : Message : You are a kid!
- ii) Age between 16 and 40 : You are young!
- iii) Age above 40 : You are old!

Solution : For developing this application we will create an HTML form for inputting the age value and then invoke the JSP page on which the appropriate messages are given based on the age.

Step 1 :

InputAge.html

```
<html>
<head>
</head>
<body>
<form method="get" action="AgeMessage.jsp">
<strong>Enter Age:</strong>
<br/>
<input type="text" name="Age">
<br/>
<input type="submit" value="Enter">
</form>
</body>
</html>
```

Step 2 : The JSP code is as follows -

AgeMessage.jsp

```
<html>
<body>
<%@ page language="java" contentType="text/html" %>
<%@page import="java.io.*" %>
<%
    String msg="";
    String age;
    int age_val;
    age=request.getParameter("Age");
    age_val=Integer.parseInt(age);
    if(age_val<15)
        msg+="You are a kid!";
    if((age_val>16)&&(age_val<40))
        msg+="You are young!";
    if(age_val>40)
        msg+="You are old!";
    out.println("<h3>"+msg+"</h3>");
%>
</body>
</html>
```

Step 3 : The output will be as follows -



Example 4.3.4 Write a JSP scriptlet for displaying even numbers between 1 to 50.

SPPU : April-18, In Sem, Marks 5

Solution :

```
<html>
<body>
<%@ page language="java" contentType="text/html"%>
<%@ page import="java.io.*"%>
<%
    for(int i=1;i<=50;i++)
    {
        if(i%2==0)
            out.println("<p>"+i+"</p>");
    }
%>
</body>
</html>
```

4.3.7 Actions and Templates

- The standard actions are those actions that can be defined by the JSP specification itself.
- Following is a list of standard actions.

Action Element	Description
<jsp:useBean>	This tag is used to instantiate the object of java bean.
<jsp:setProperty>	This tag is used to set the property value for the java bean.
<jsp:getProperty>	This tag is used to get the property value from the java bean.
<jsp:include>	This tag works as a subroutine. It includes the response from servlet or JSP when request is being processed.
<jsp:param>	For adding the specific parameter to the request this tag is used. Normally this tag is used inside <jsp:include> or <jsp:forward>
<jsp:forward>	This tag helps in forwarding the current request to servlet or to JSP page.
<jsp:plugin>	This tag is used to generate HTML code and to embed the applet into it.
<jsp:attribute>	This tag sets the value of the action attribute
<jsp:element>	This tag generated the XML elements dynamically.
<jsp:text>	This tag is used to handle template text. When JSP pages are written as XML documents then this tag is used.
<jsp:body>	This tag is used to set the body element.

Let us now discuss the standard actions by creating appropriate JSP pages.

Include action

Using the **<jsp:include>** action we can include different files in current jsp pages.

For example -

Include_action.jsp

```
<html>
  <head>
    <title> Include action Demo </title>
  </head>
  <body>
    <h1>This page contains include action</h1>
    <jsp:include page="test_include.html" />
    <jsp:include page="test_jsp.jsp" />
  </body>
</html>
```

Now the static html page can be

test_include.html

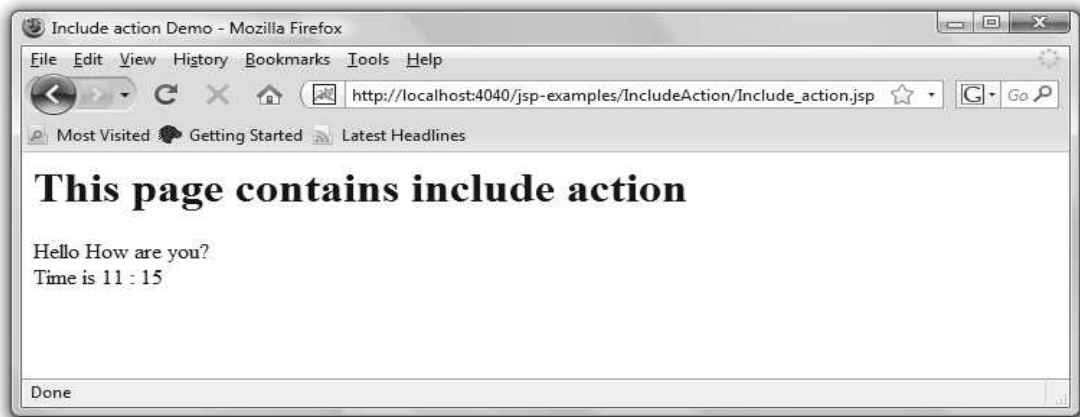
```
<html>
<body>
    Hello How are you?
</body>
</html>
```

Now the JSP page for displaying dynamic content is

test_jsp.jsp

```
<html>
  <body>
    <br>Time is <%= new java.util.Date().getHours()%>
    : <%= new java.util.Date().getMinutes()%>
  </body>
</html>
```

For getting the output, open the web browser and open our main JSP page **Include_action.jsp**, it is as follows-



Forward action

- Using forward action we can forward the current request to corresponding JSP page or servlet.
- Here is a simple demonstration in which we have created two web pages the first one is a main JSP page named as first.jsp in which we are using JSP action forward. And forwarding the current request to an HTML page which is named as Second.html.

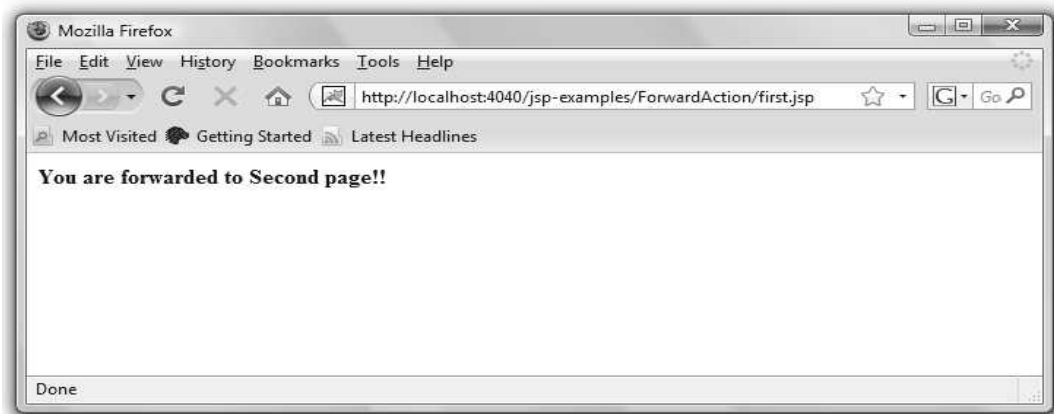
first.jsp

```
<html>
  <head>
    <title> Forward action Demo </title>
  </head>
  <body>
    <h1>This page contains forward action</h1>
    <jsp:forward page="Second.html" />
  </body>
</html>
```

Second.html

```
<html>
<body>
  <strong>You are forwarded to Second page!!</strong>
</body>
</html>
```

For getting the output open the web page and open the main JSP page that is **first.jsp** and you will get the following output -

Output**useBean action**

- Java beans are reusable components. We can use simple Java bean in the JSP.
- Beans are used in the JSP pages as the instance of class. We must specify the scope of the bean in the JSP page. Here scope of the bean means the range time span of the bean for its existence in JSP.
- Bean is basically a Java class with get and set methods.
- Let us understand the useBean action with illustrative example.

Example 4.3.5 Write a JSP document to display number of times user is visiting particular page.

Solution :

Step 1 : Create a simple bean in which a variable count is maintained. This count can be incremented using the JSP page.

CounterDemo.java

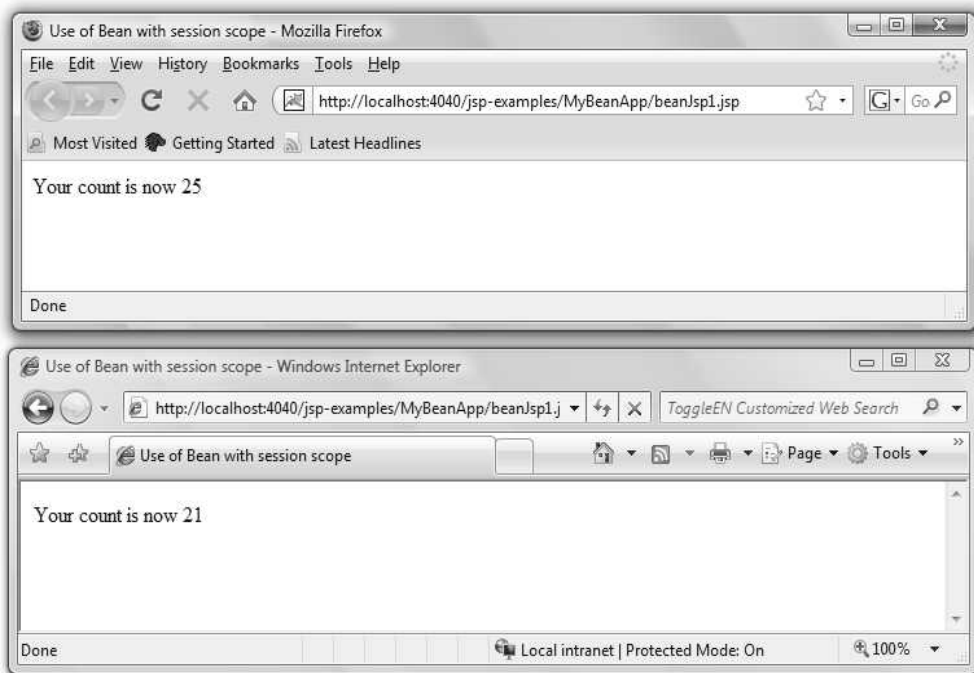
```
public class CounterDemo
{
    public int count;
    public CounterDemo()
    {
        count=0;
    }
    public int getCount()
    {
        return count;
    }
    public void setCount(int count)
    {
        this.count=count;
    }
}
```

Step 2 : Now create a JSP page which make use of the bean created in step 1 in the session scope.

beanJsp1.jsp

```
<html>
<head>
<title> Use of Bean with session scope </title>
</head>
<body>
<jsp:useBean id="bean_id" class="CounterDemo" scope="session" />
<%
bean_id.setCount(bean_id.getCount()+1);
%>
Your count is now <%= bean_id.getCount() %>
</body>
</html>
```

Step 3 : Open two different web browsers Mozilla Firefox and Internet Explorer and open the same JSP page in both the browsers, repeatedly refresh the JSP page in both the browsers and observe it.



Example 4.3.6 Write the differences between include action and include directive in JSP.

SPPU : Dec.-18, May-19, End Sem, Marks 5

Solution : The difference is as follows -

- 1) Include directive includes the file at translation time - that is when the JSP gets converted into the equivalent servlet whereas the include action includes the file at runtime.
- 2) If the included file is changed but not the JSP which is including it then the changes will reflect only when we use include action tag. The changes will not reflect if you are using include directive as the JSP is not changed so it will not be translated for request processing and hence the changes will not reflect.
- 3) Syntax difference : Include directive : `<%@ include file="file_name" %>` whereas include action has like this `<jsp:include page="file_name" />`
- 4) When using include action tag we can also pass the parameters to the included page by using param action tag but in case of include directive it's not possible.

Review Questions

1. What are the usage of JSP directives and JSP actions.

SPPU : April-18, Marks 5

2. Write JSP program to demonstrate use of page directive, scriptlet and expression tag.

SPPU : Dec.-19, Marks 5

4.4 Implicit Objects

SPPU : May-18, 19, Dec.-18, Marks 5

- The implicit objects are predefined variables used to access request and application data. These objects are used by the scripting elements. Following is a list of implicit objects.

Variable name	Class/Interface name	Meaning	Sample methods
application	javax.servlet.ServletContext	This object provides the resources shared within a web application.	log() getServerinfo()
config	javax.servlet.ServletConfig	It helps in passing the information to servlet or JSP page during initialization.	getInitParameter() getServletName()
request	javax.servlet.http.HttpServletRequest	It provides the methods for accessing the information made by current request.	getContentType() getLocalAddr() getServerName()
response	javax.servlet.http.HttpServletResponse	It provides the methods related to adding cookies, sessions or setting header.	addCookie() addHeader() flushBuffer() getContentType() setContentType()

out	javax.servlet.jsp.JspWriter	It provides the methods related to I/O	clear() newLine()
page	java.lang.Object	This variable is assigned to instance of JSP implementation class. It is rarely used variable.	
pageContext	javax.servlet.jsp.PageContext	It provides access to several JSP page attributes	getPage() getRequest() getResponse() getSession()

Review Questions

1. Create a simple JSP pages which will demonstrates use of implicit session objects in JSP.

SPPU : May-18, Dec.-18, Marks 5

2. List and elaborate any five JSP implicit objects.

SPPU : May-19, Marks 5

4.5 JavaBeans Classes and JSP

SPPU : May-18, Marks 5

- Java beans are reusable components. We can use simple Java bean in the JSP.
- Beans are used in the JSP pages as the instance of class. We must specify the scope of the bean in the JSP page. Here scope of the bean means the range time span of the bean for its existence in JSP.
- Bean is basically Java class with **getXX** and **setXX** methods.

Characteristics for JavaBean Class

- 1) Java Bean class contains a default **no argument constructor**. This constructor must have the access specifier as **public**.
- 2) It should be serializable and should **implement serializable interface**.
- 3) It can have any number of properties which can be read or written. For these properties we write **getter and setter methods**.

JavaBean Properties

- A JavaBean Property is a named attribute that can be accessed using the object.
- The JavaBean properties can be accessed through two methods namely
 - 1) **getPropertyName()** : This method is called **accessor** method. It returns the property value. This method is written in some special way. For example - If the property is **rollNo** then the method name for **rollNo** is **getRollNo()**
 - 2) **setPropertyName()** : This is called **mutator** method. This method is generally used to assign the value to the properties. This method is written in some special way. For example - If the property is **rollNo** then method name will be **setRollNo()**

The read-only attribute is for **getPropertyName()** method and write-only attribute is for **setPropertyName()** method.

Example : Bean program using JSP for Employee Information.

Step 1 : Create a Bean program in some package. I have created a folder(as a package) named **BeanProg**. The code for JavaBean class is written as follows -

Employee.java

```
package BeanProg;
public class Employee implements java.io.Serializable {
    private String empName=null;
    private int empNo=0;
    public Employee() { //no argument constructor
    }
    public String getEmpName() {
        return empName;
    }
    public int getEmpNo() {
        return empNo;
    }
    public void setEmpName(String empName) {
        this.empName=empName;
    }
    public void setEmpNo(int empNo) {
        this.empNo=empNo;
    }
}
```

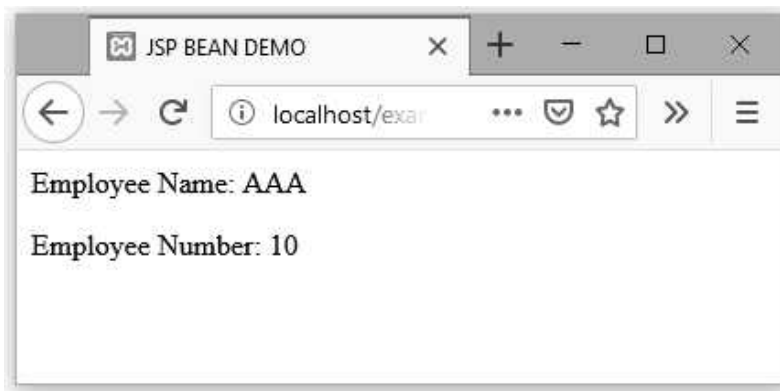
Step 2 : The JSP code can be written as follows -

EmpBeanDemo.jsp

```
<html>
  <head>
    <title> JSP BEAN DEMO </title>
  </head>
  <body>
    <jsp:useBean id="empID" class="BeanProg.Employee">
      <jsp:setProperty name = "empID" property = "empName" value = "AAA"/>
      <jsp:setProperty name = "empID" property = "empNo" value = '10'/>
    </jsp:useBean>
    <p>Employee Name:
    <jsp:getProperty name = "empID" property = "empName"/>
    </p>
    <p>Employee Number:
    <jsp:getProperty name = "empID" property = "empNo"/>
    </p>
  </body>
</html>
```

Step 3 : Start tomcat and open the JSP page in Web browser you will get the output as follows -

Output



Example 4.5.1 Write a client server JSP program to find the simple interest and display the result in the client.

Solution : This program is created using following steps.

Step 1 : Create an simple HTML form for inputting the values of P, N and R. The HTML document for this is as given below.

Input.html

```

<html>
<head><title>Input Form</title></head>
<body>
Enter following values:<br/>
<form method="post" action="interest.jsp">
Amount:<input type="text" name="amount" value="" size="10"/>
Period:<input type="text" name="period" value="" size="3"/>
Rate of Interest:<input type="text" name="rate" value="" size="3"/>
<input type="submit" value="Submit"/>
</form>
</body>
</html>

```

Step 2 : Create JSP page which acts as a client to receive the values for P,N and R. These values will be sent to the server Java program for calculating the simple interest. The calculated interest value is displayed by this same JSP page. The code for this is as follows -

Interest.jsp

```

<html>
<head>
<title> Simple Interest Calculation Demo </title>
</head>
<body>
<jsp:useBean id="bean_id" class="interestcalcDemo.InterestDemo" scope="session" />
<% String s_p=request.getParameter("amount");
int p=Integer.parseInt(s_p);
bean_id.setP(p);

String s_n=request.getParameter("period");
int n=Integer.parseInt(s_n);
bean_id.setN(n);

String s_r=request.getParameter("rate");
int r=Integer.parseInt(s_r);
bean_id.setR(r);
%>
</form>
Amount = <%= p%><br/>
Period = <%= n%><br/>
Rate = <%= r%><br/>
<strong>Interest is now <%= bean_id.getI() %><br/></strong>
</body>
</html>

```

Calling server Java program for computing the simple interest.

Collecting values from **input.html** created in **Step 1** and sending them to sever program using method **set**

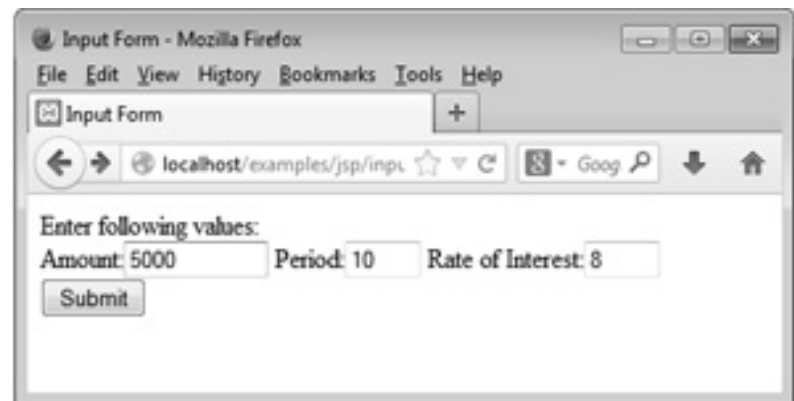
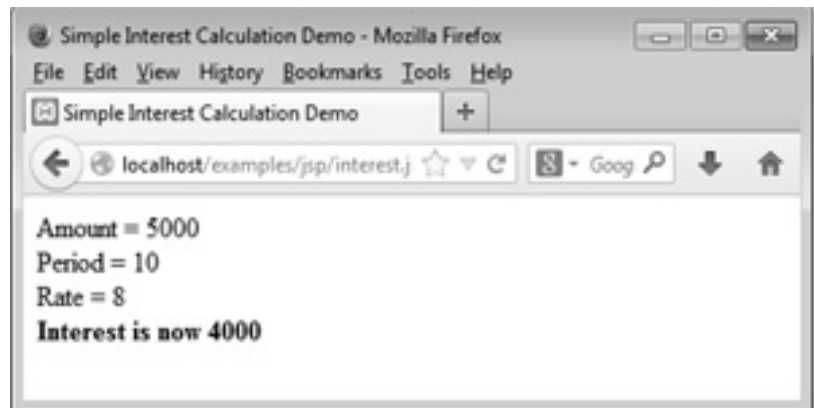
Obtaining interest value from server

Step 3 : Following is a Java bean program which acts as a Server. It calculated the simple interest using the formula $(p*n*r)/100$. This value is then returned to the client JSP program created in Step 2. The code for server java program is as follows -

InterestDemo.java

```
package interestcalcDemo;
public class InterestDemo
{
    public int p,n,r;
    public InterestDemo()
    {
        p=0;n=0;r=0;
    }
    public int getI()
    {
        int i=(p*n*r)/100;
        return i;
    }
    public void setP(int p)
    {
        this.p=p;
    }
    public int getP()
    {
        return p;
    }
    public void setN(int n)
    {
        this.n=n;
    }
    public int getN()
    {
        return n;
    }
    public void setR(int r)
    {
        this.r=r;
    }
    public int getR()
    {
        return r;
    }
}
```

Output



Review Question

1. What is the JSP action tags ? Discuss in details `jsp:useBean` action tag.

SPPU : May-18, Marks 5

4.6 Support for the Model-View-Controller Paradigm

SPPU : Dec.-18, Marks 6

- The design model of JSP application is called MVC model. The MVC stands for Model-View-Controller. The basic idea in MVC design model is to separate out design logic into three parts - modelling, viewing and controlling.
- Any server application is classified in three parts such as business logic, presentation and request processing.
- The **business logic** means the coding logic applied for manipulation of application data. The **presentation** refers to the code written for look and feel of the web page. For example: background color, font style, font size, placing of controls such as text boxes, command buttons and so on. The request processing is nothing but a combination of business logic and presentation. The request processing is always done in order to generate the response.

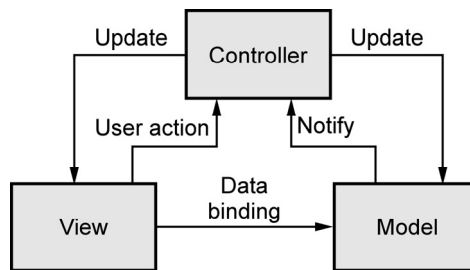


Fig. 4.6.1 Model View Controller (MVC)

- According to the MVC design model, the model corresponds to business logic, view corresponds to presentation and controller corresponds to request processing.

Advantages of using MVC design model

- The use of MVC architecture allows the developer to keep the separation between business logic, presentation and request processing.
- Due to this separation it becomes easy to make **changes** in presentation without disturbing the business logic. The changes in presentation are often required for accommodating the new presentation interfaces.

Review Question

1. What is MVC ? Draw and explain MVC architecture for developing web applications.

SPPU : Dec.-18, Marks 6

Part II : Web Services**4.7 Web Services Concepts**

SPPU : May-18,19, Dec.-18, Marks 8

Definition : The Web Services are the software systems that are displayed by the web browser using the web protocol. These software systems are used by the some software applications rather than by end-users directly.

- Web service is a software system designed which is **independent** of specific hardware or software on which it is running.
- **Examples** of a web service are -
 1. **Credit card validation system** - For using this service the client simply enters the ID and password for the credit card and web service reports the validity of it.
 2. **Whether forecast system** - The clients submits the location and the web service returns the information about current whether as well as it may report some predictions.
 3. **Currency converter** - The client enter the source and destination currency along with the amount and appropriate converted information will be displayed.

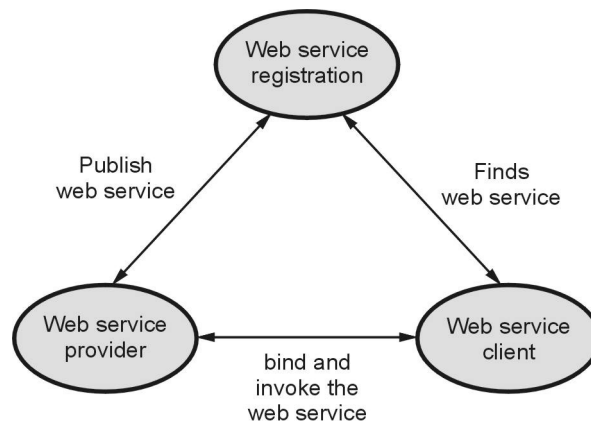


Fig. 4.7.1 Basic web service model

Following are the steps that a web service model follows -

Step 1 : In the first step, a service provider **publishes** a web service in the **web service registry**.

Step 2 : A web client who demands for some web service **searches** in the registry. After finding a match for the desired web service in the registry. After finding a match for the desired web service in the registry, the client chooses it.

Step 3 : The client **binds** to the corresponding web service provider and **invokes** for the service.

4.7.1 Components of Web Services

The role of various components such as XML, SOAP, WSDL, UDDI in web service building is as shown below -

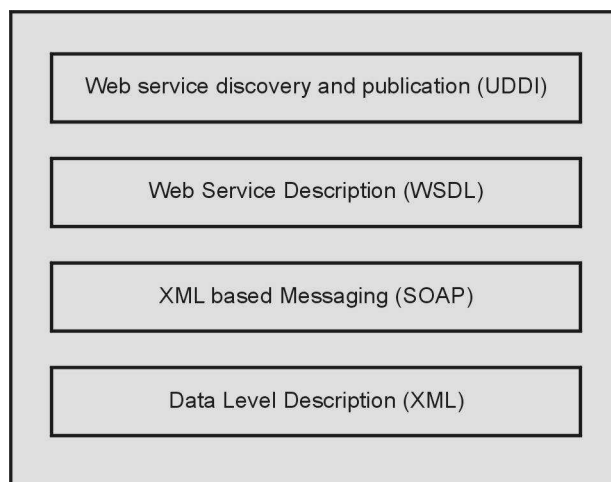


Fig. 4.7.2 Web service elements

- At the lowest level of web services there are simply XML files that are for data level description.
- SOAP - **S**imple **O**bject **A**ccess **P**rotocol is a simple XML based protocol which allows exchange of messaging over HTTP.
- Typically web services make use of this protocol for exchange of information .
 - The SOAP makes use of JAX-RPC component. These are **J**AVA **A**PI for **X**ML that makes use of **R**emote **P**rocedure **C**alls.
 - Basically SOAP defines message structure, encoding rules and conventions for exchange of information in RPC programming model.
 - The following table describes core JAX-RPC interfaces and classes.

Interface / Class	Description
Service	Main client interface. Used for both static and dynamic invocations.
Service Factory	This is a factory class which is used to create services
Stub	It is used by client for invoking the web services.
Call	For invoking the web services the call is used.
JAXRPCException	This exception is thrown when an error occurs while invoking web service.

- At the higher level very important element of web service lies i.e. WSDL. The **Web Service Description Language** is used for abstract description of web services. This description is provided in XML document. When the client wants to make use of web service, the users of the client make read this description file and understand what input data is needed to perform the web service. Such a special file is called WSDL i.e. Web Services Description Language.
- UDDI i.e. **Universal Description and Discovery Integration**, provides a directory of web services, so that client can easily discover the services of their choices. The UDDI maintains a registry in which the web services are registered.
- In this chapter we will learn how to develop a simple web service using JAX-RPC. Further we will learn how to build a **web client** for using this service.

Review Questions

1. *Identify and justify benefits of using web services ?*

SPPU : May-18, Marks 6

2. *What are web services ? List and discuss components of web services.*

SPPU : Dec.-18, Marks 6

3. *What are the web services ? List and explain layers in protocol stack of web service architecture.*

SPPU : May-19, Marks 8

4.8 Writing a Java Web Service

In this section, we will create two Web Projects – one for Server and other for Client.

The Server defines the web service and client invokes this web service

We can develop a web service using Apache Axis2 and Eclipse.

Prerequisite :

- 1) JDK must be installed.
- 2) Eclipse must be installed
- 3) Apache Server, Tomcat must be installed. Install the software package XAMPP for this purpose.

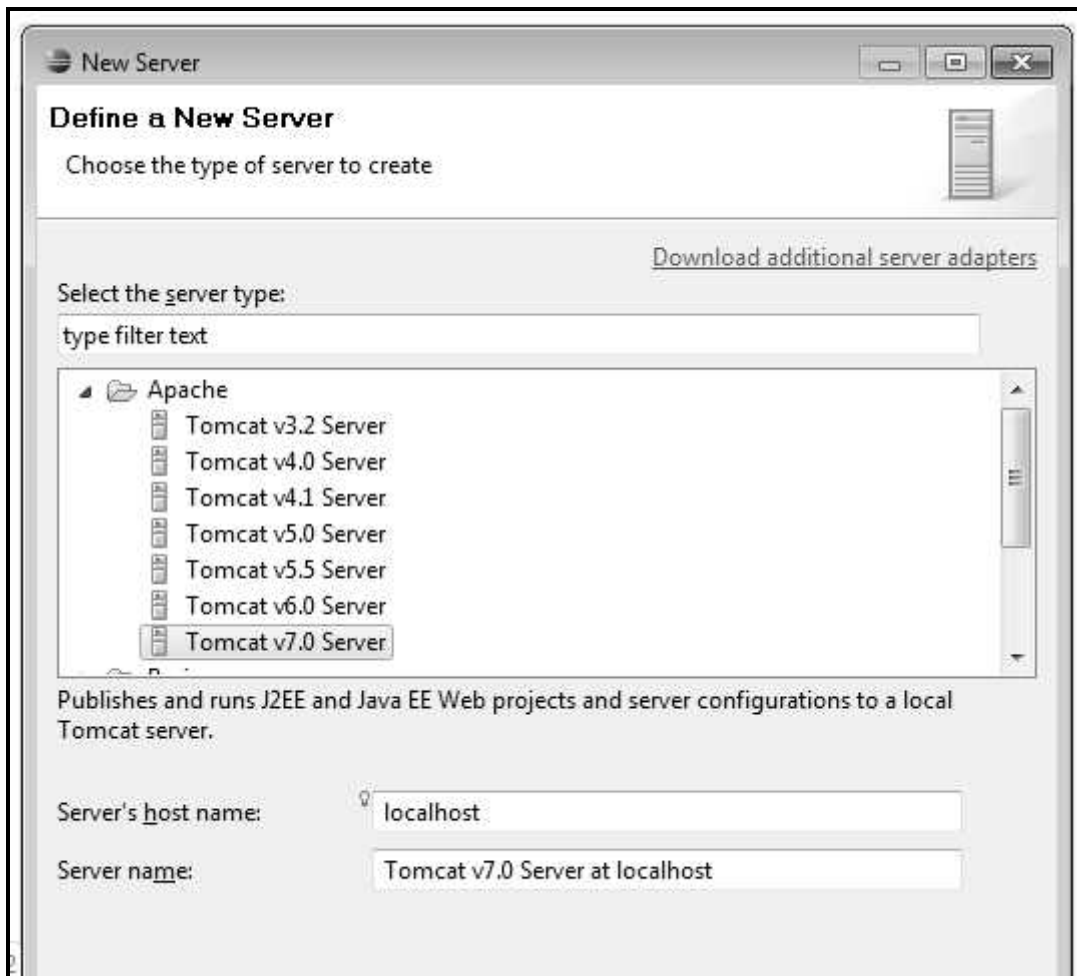
Configuring Tomcat to Eclipse

Step A : To integrate Tomcat Server with your Eclipse, Select **File->New->Other...** from Eclipse main menu.

Step B : Select Server->Server and click Next



Step (3) Select Tomcat Version XX and click **Next**.



Step C : Browse to the folder of Tomcat. If XAMP is installed then it is
c:\XAMPP\tomcat

Step D : Click **Finish**

4) Install Axis2.

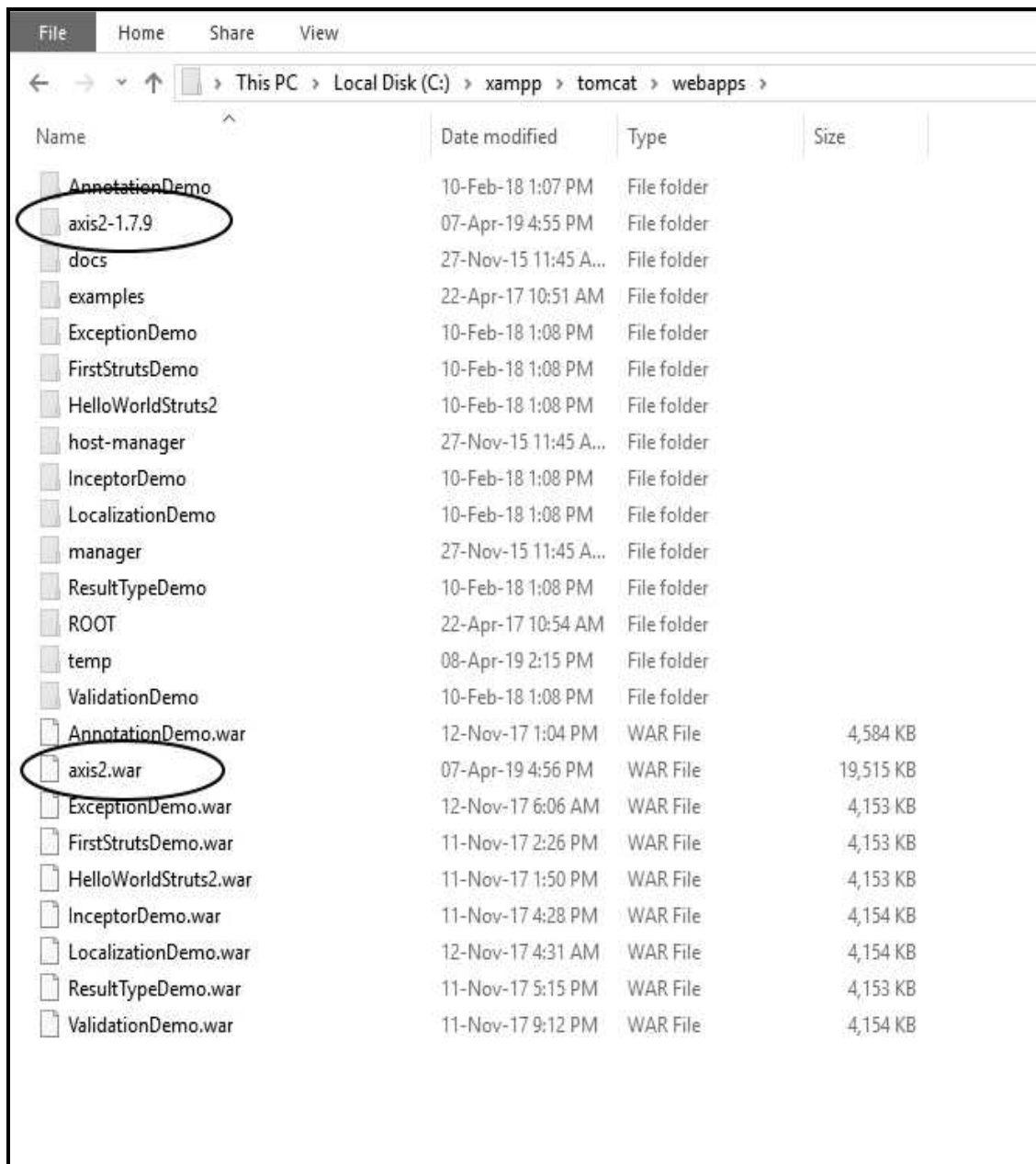
Installation of Axis 2 :

Step 1 : Open the web browser and get the axis2 package using following URL

<http://axis.apache.org/axis2/java/core/download.html>

Step 2 : Click on Binary Distribution Link and download axis2-1.7.9-bin.zip. Also download axis2-1.7.9-war.zip

Step 3 : Unzip both the folders and copy these folders at your c:\tomcat-directory\webapps folder. Following screenshot helps in understanding the location



Configuring Axis2 in Eclipse

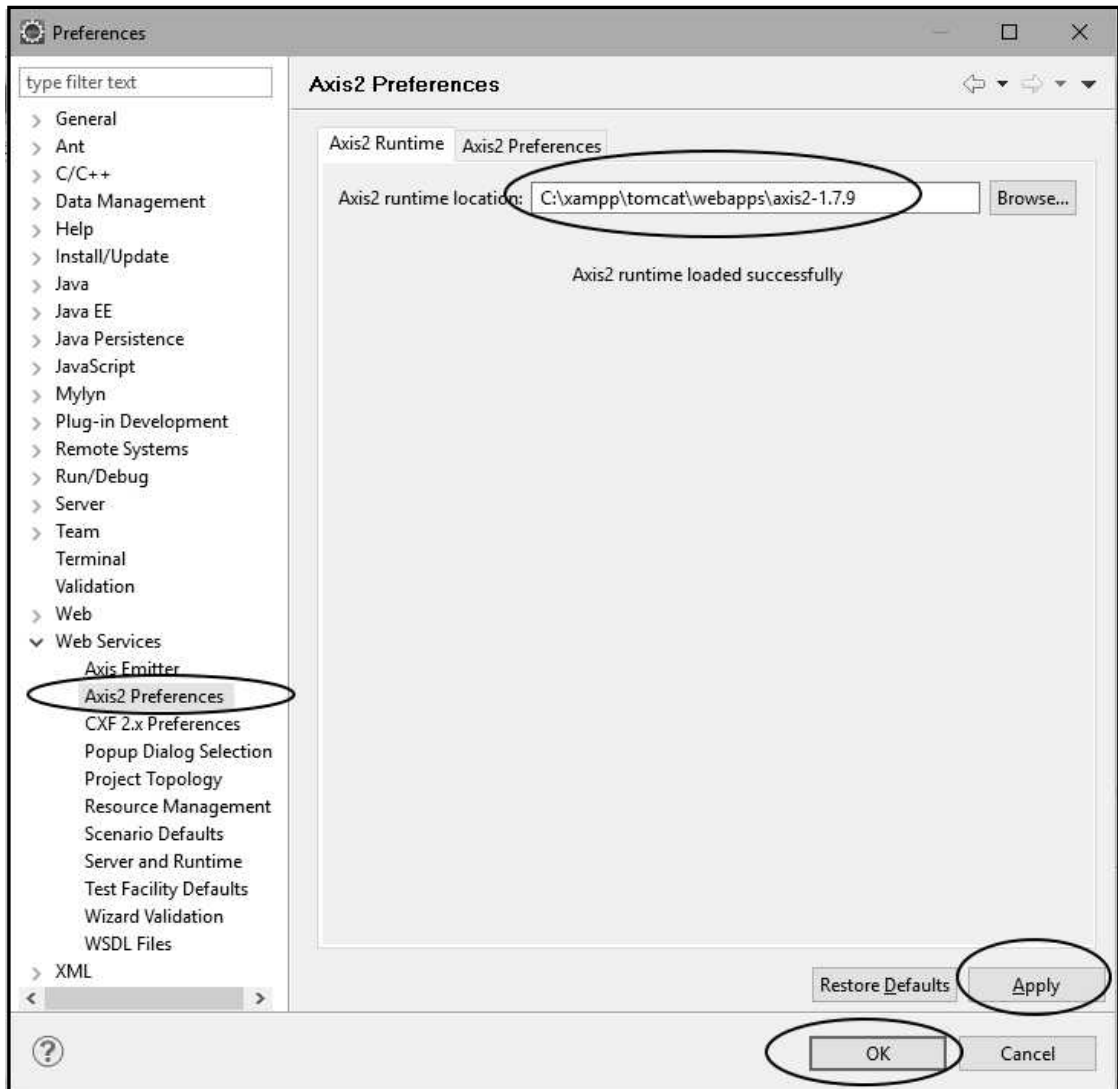
Step 1 : Open the Eclipse.

Step 2 : Click on **Window -> Preferences.**

Step 3 : Locate **Web Services.** Click **Axis2 Preferences.**

Step 4 : Set the location of Axis2 folder with the help of **Browse..** button. It is your Webapps folder in Tomcat directory. On successful deploying of Axis2, we get “Axis2 runtime loaded successfully” message. Click **Apply** and then **Ok** button.

Following screenshot illustrates it.

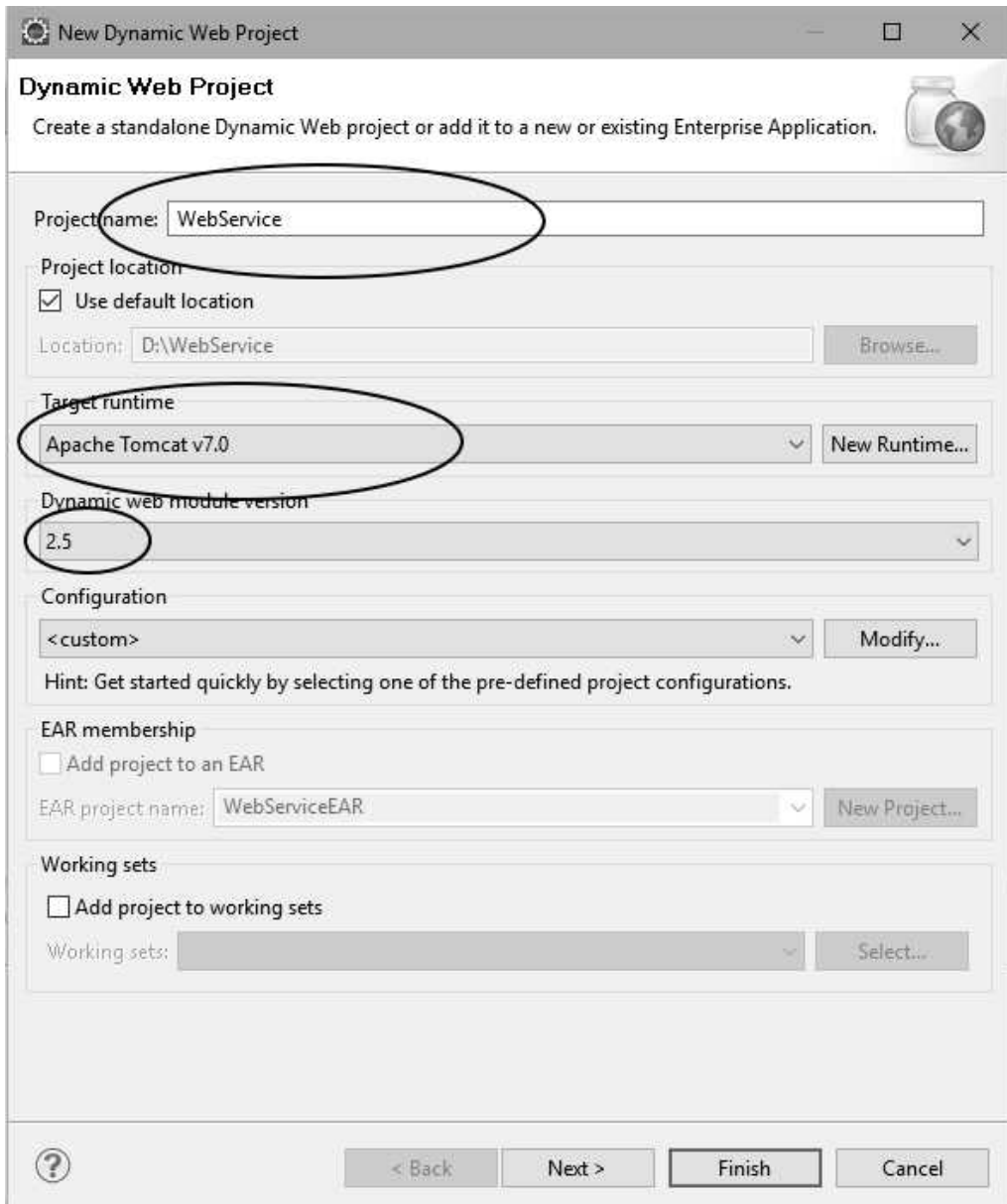


Let us now discuss step by step - how to create a simple Web Service.

Example 4.8.1 *Create a web service in java environment to return the sum of two integers with necessary deployment procedures.*

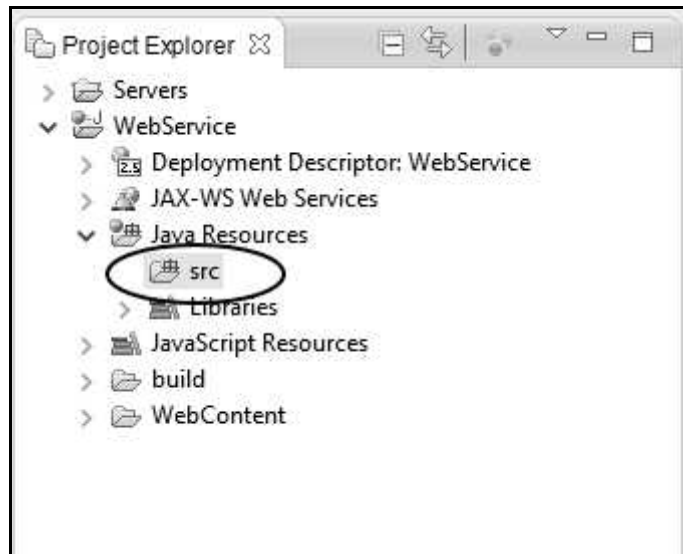
Solution :**Stage I : Creating Server**

Step 1 : Click on Eclipse icon. Click on File->New->Dynamic Web Project. Give the suitable name to the project. I have given the project name as **WebService**. Set the Tomcat Web Server and appropriate version number. Following screenshot illustrates this.

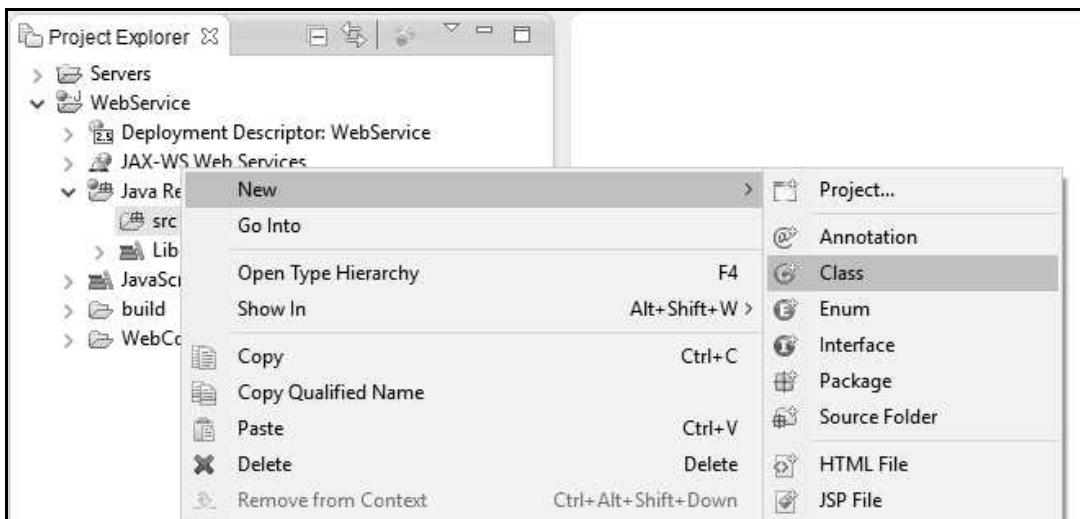


Click **Finish** button.

Step 2 : Locate **src** folder under the project in WebService project. The snapshot of Project Explorer is as shown below



Right click on **src** folder Select **New-> Class**



A New Class Window opens up. Give suitable Package name, Class name and uncheck the check box for void main function. Then click **Finish** button. It is illustrated by following screenshot.

New Java Class

Create a new Java class.

Source folder: [Browse...](#)

Package: [Browse...](#)

☐ Enclosing type: [Browse...](#)

Name:

Modifiers: ☒ public ☐ default ☐ private ☐ protected
☐ abstract ☐ final ☐ static

Superclass: [Browse...](#)

Interfaces: [Add...](#) [Remove](#)

Which method stubs would you like to create?

☒ `public static void main(String[] args);`
☐ Constructors from superclass
☒ Inherited abstract methods

Do you want to add comments? (Configure templates and default value [here](#))

☐ Generate comments

[Finish](#) [Cancel](#)

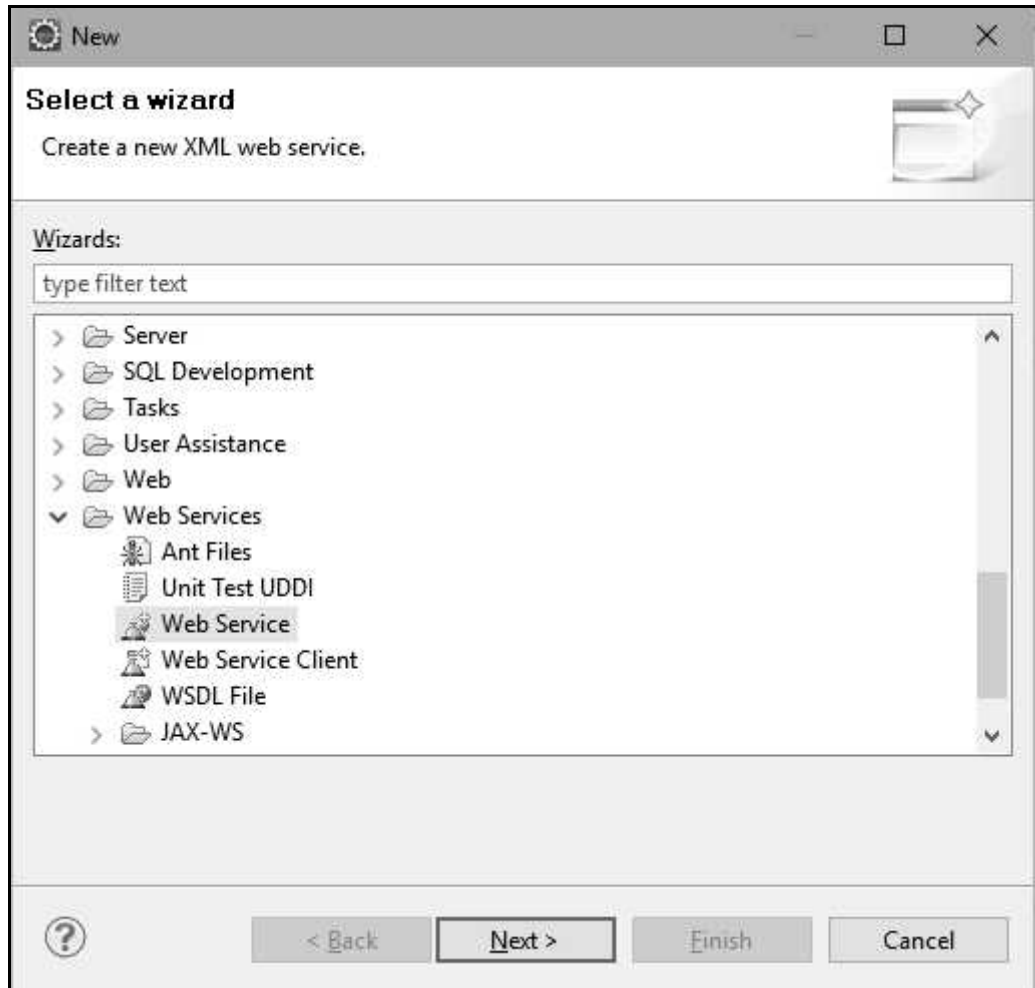
Step 3 : Now write the following code for **MyServer.java** .

```
package com.mypackage;  
  
public class MyServer {  
    public int add(int a,int b){
```

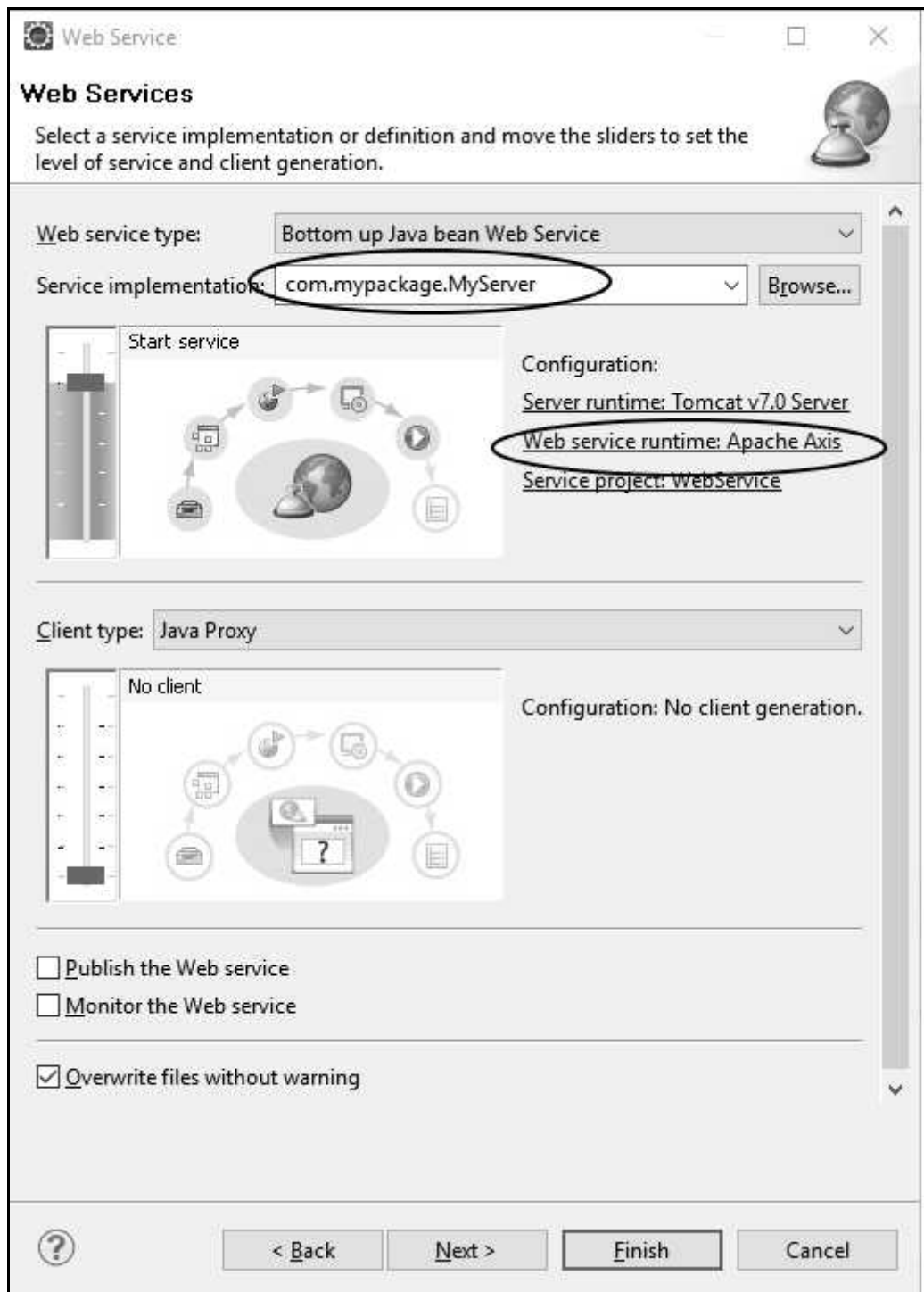
```
    return a+b;
  }
}
```

Save the above code.

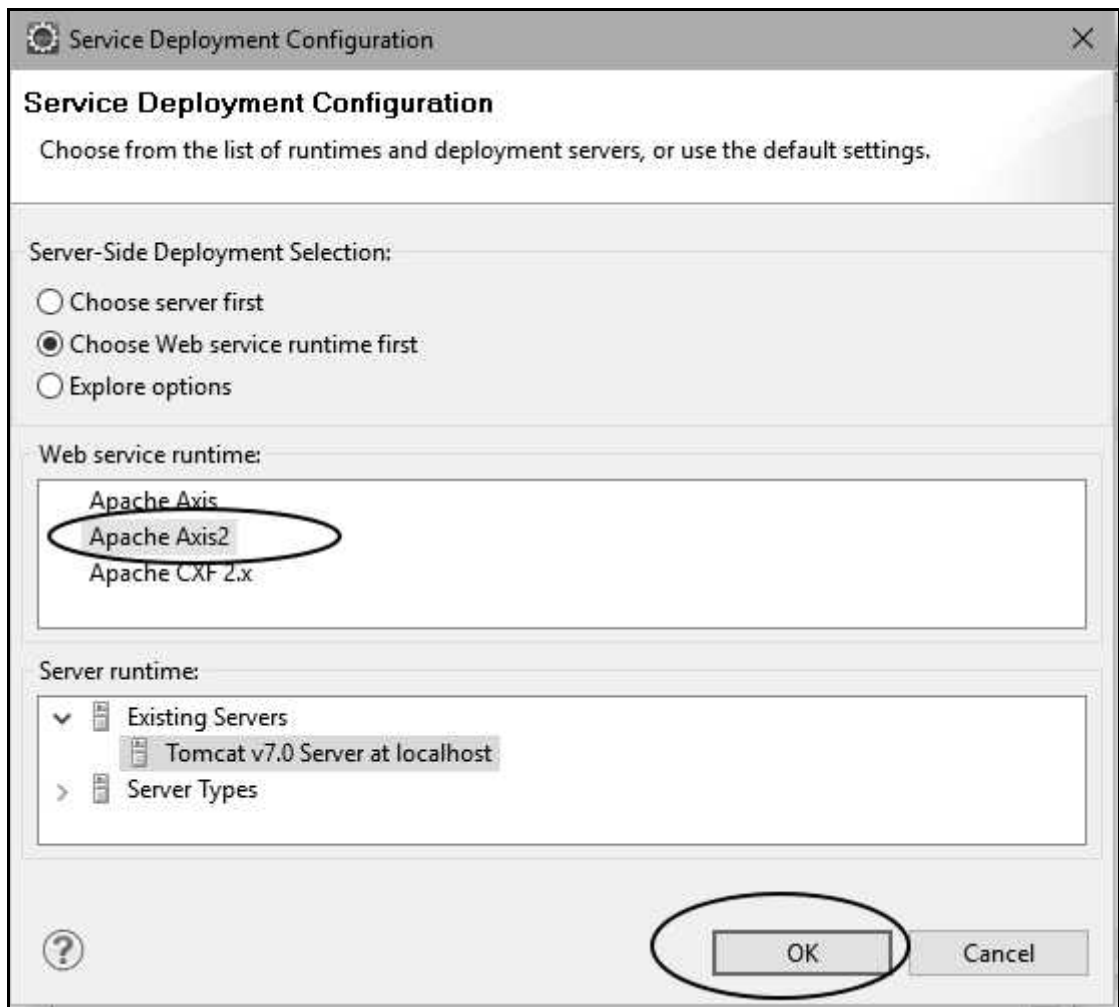
Step 4 : Now right click on **src** folder. Click **New->other->Web Services->Web Service**. Then click **Next** button.



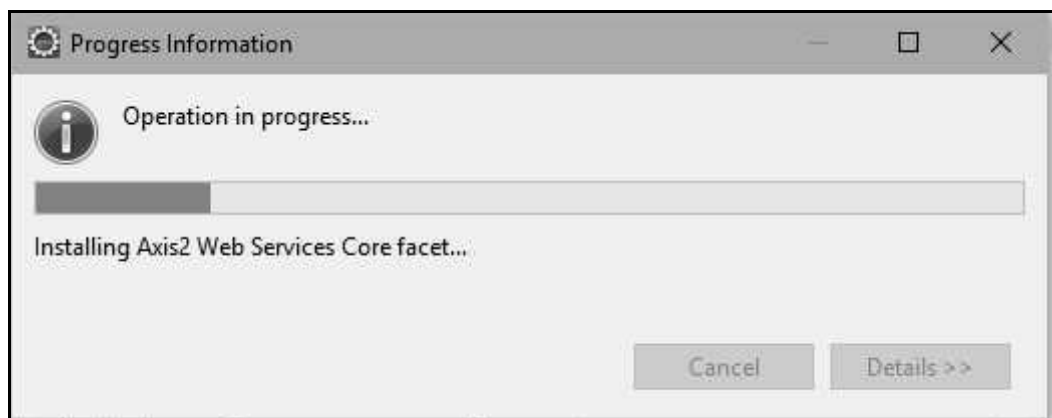
The **Web Services** window will appear. Write the name of service implementation and click on **Web service runtime:Apache Axis**. Following screenshot shows this -



Click on **Apache Axis2** and then click **Ok** button. Refer following screenshot.



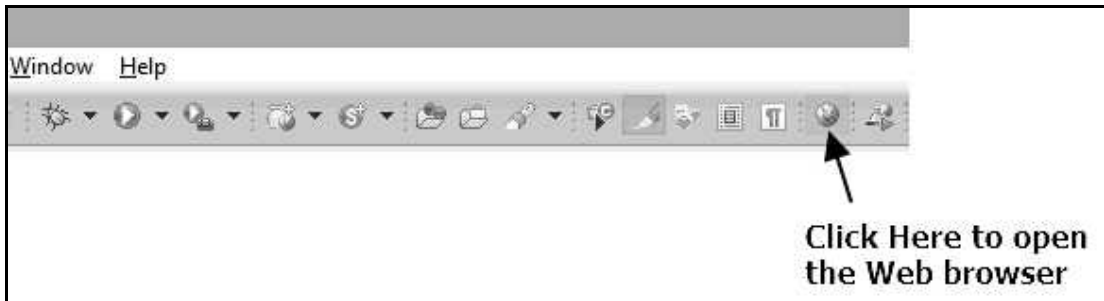
Then click on Finish button. You will get the progress window as,



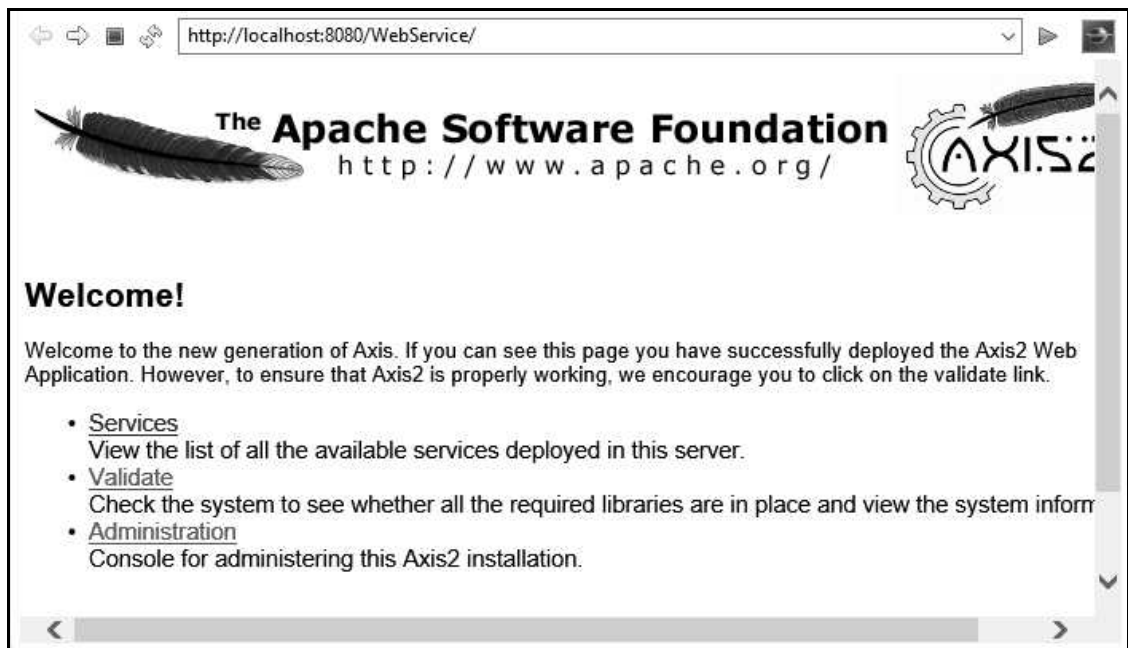
This will generate the WSDL file for your application.

Step 5 : To view the generated WSDL file we need to open up the Web Browser in Eclipse. For that

1. Open Web Browser present on the Toolbar



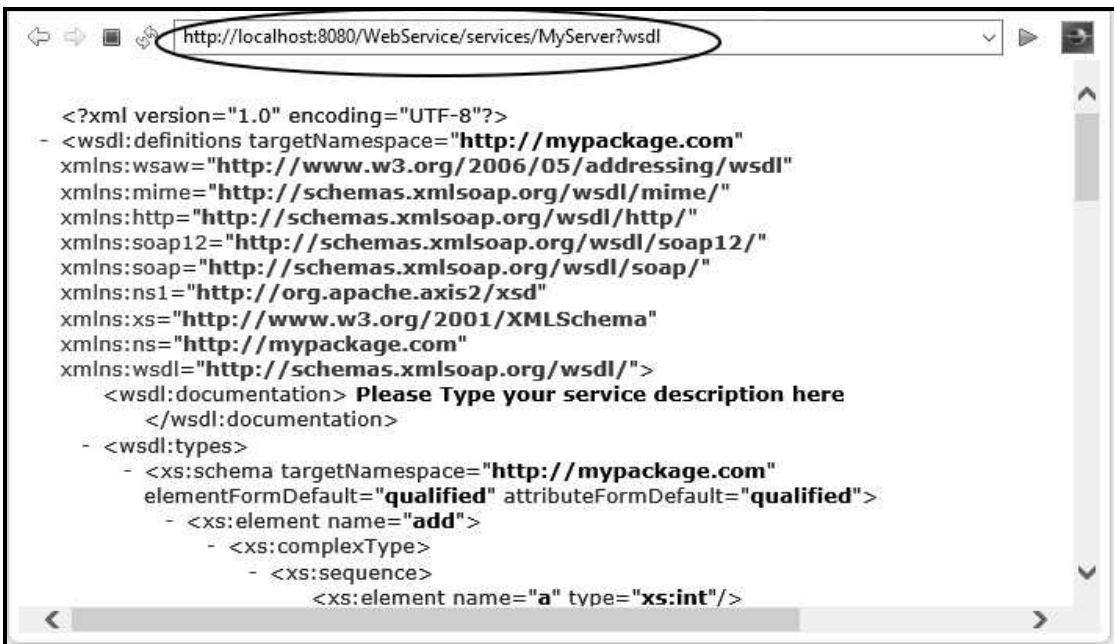
2. Type the URL as follows -



3. Now click on **Services**. Your Web Service name **MyServer** can be located on the page.



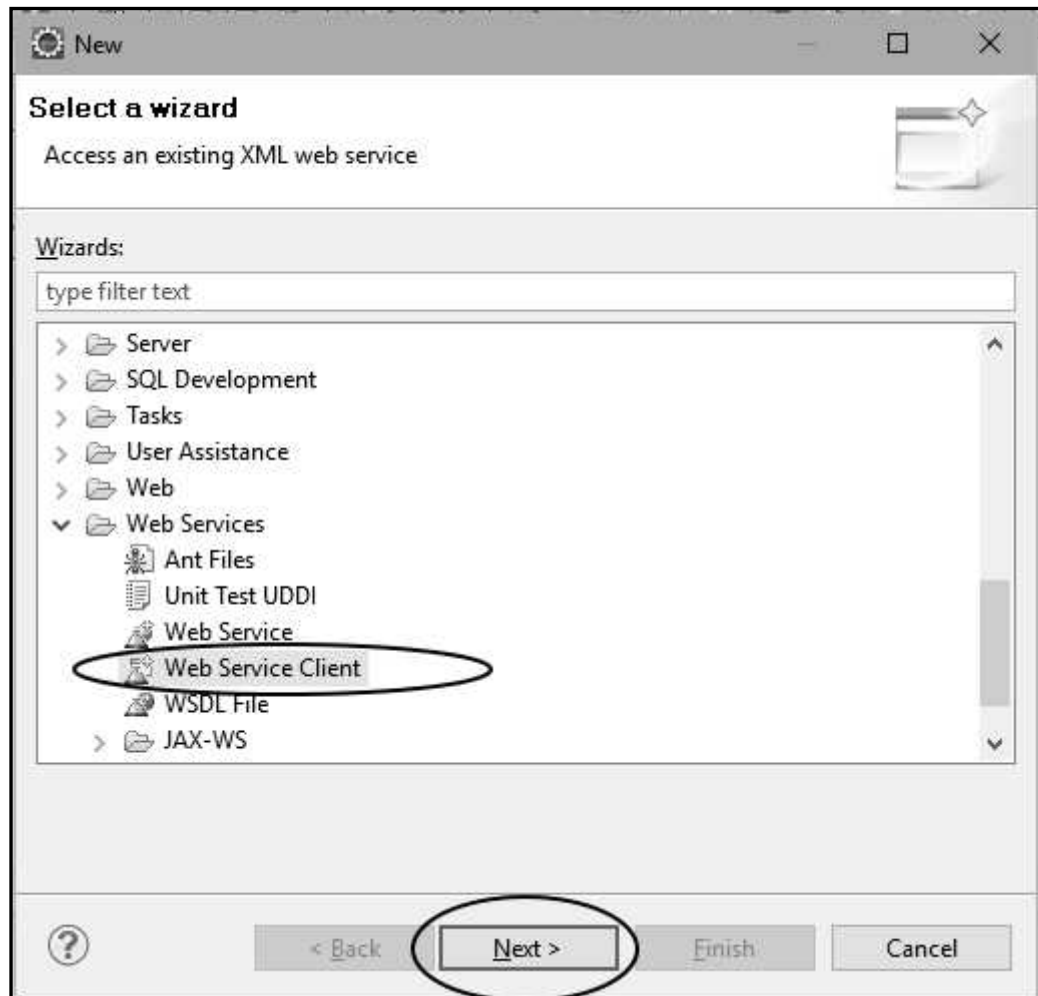
4. Just click on **MyServer** link another web page will be opened up in the browser. This is actually your WSDL file. Copy the full URL for WSDL file marked in screenshot given below. Save this URL in some Notepad file. Note that: We need this URL during the creation of Client.



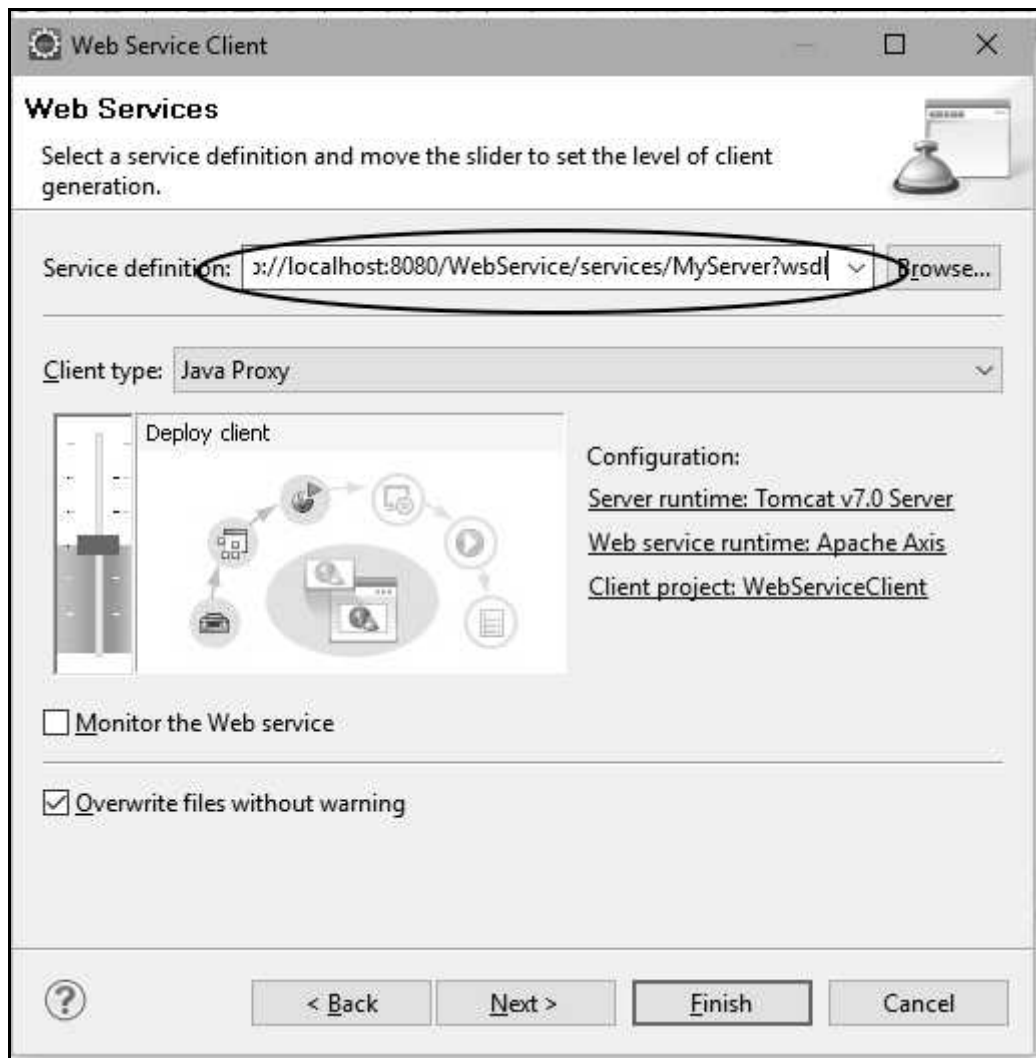
Stage II : Creating Client for Web Service

Step 6 : Now go to **File->New->Dynamic Web Project**. Give suitable name for your application. Here we are creating a client for the Web service created in above steps. Hence I have given the name **WebServiceClient**. Then click **Finish** button.

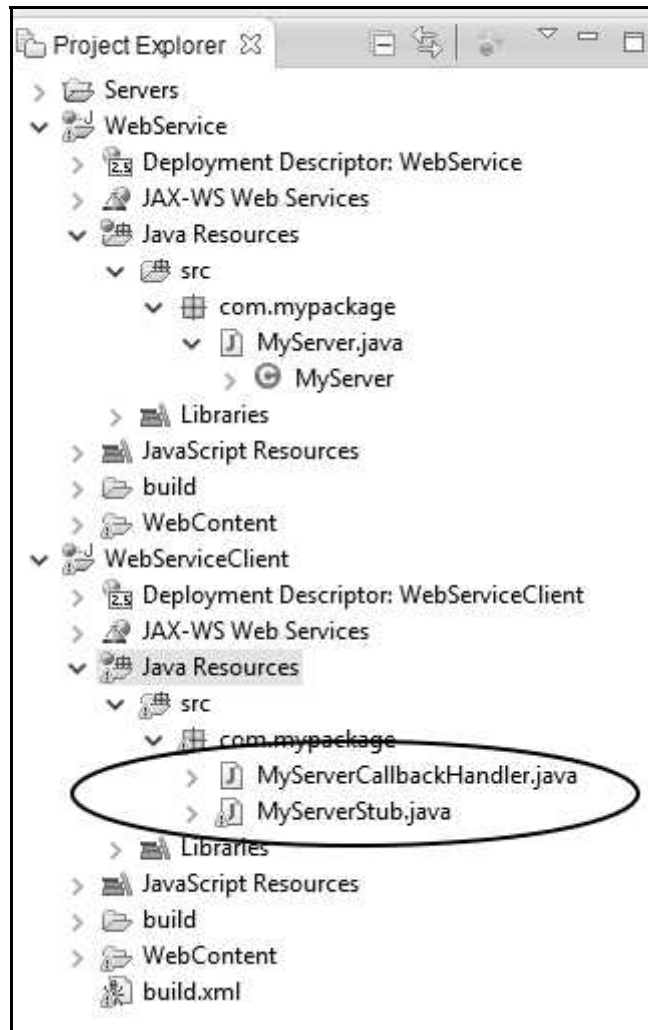
Step 7 : Now under **WebServiceClient**, right click on **src** folder. Click on **New->Other->Web Services->Web Service Client**



Click the **Next** button. Now on **Web Services** window. Copy the URL of WSDL which you have copied in Step 5(4).



Then Select **Web Service runtime:Apache Axis**. Set it to **Apache Axis2** click **OK** button. Then Click on **Finish** button. You will get Progress Information and then two files will get generated. These are as shown in following screenshot.



Step 8 : Now right click on **src** folder of **WebServiceClient**. Click on **New->Class**. The **New Java Class** window will appear. Set appropriate package name, class name and Check for void main method. Here is the screenshot.

New Java Class

Create a new Java class.

Source folder: [Browse...](#)

Package: [Browse...](#)

☐ Enclosing type: [Browse...](#)

Name:

Modifiers: ☒ public ☐ default ☐ private ☐ protected
☐ abstract ☐ final ☐ static

Superclass: [Browse...](#)

Interfaces: [Add...](#) [Remove](#)

Which method stubs would you like to create?

☒ public static void main(String[] args)
☐ Constructors from superclass
☒ Inherited abstract methods

Do you want to add comments? (Configure templates and default value [here](#))
☐ Generate comments

[?](#) [Finish](#) [Cancel](#)

Click on **Finish** button. You can now write the Java Client code for web service. The code is as follows,

MyClient.java

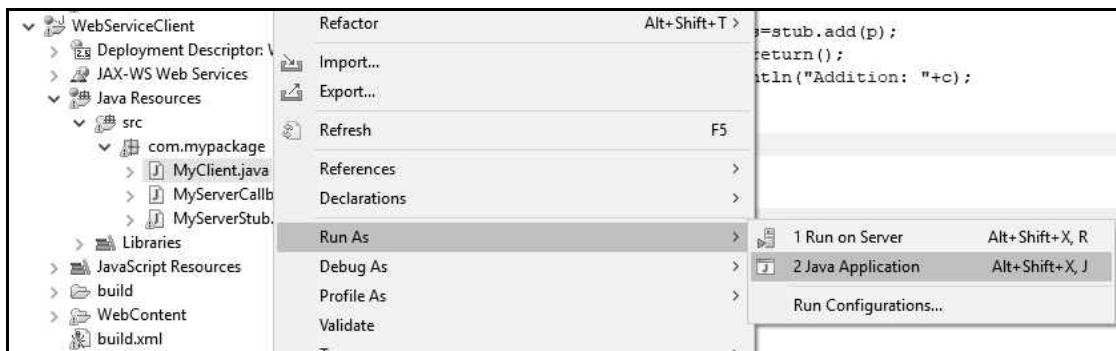
```
package com.mypackage;  
import com.mypackage.MyServerStub.Add;  
import com.mypackage.MyServerStub.AddResponse;
```

```

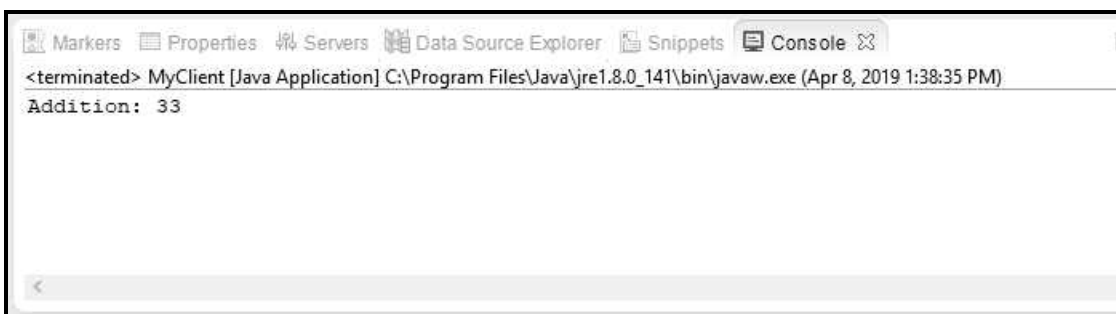
public class MyClient {
    /**
     * @param args
     */
    public static void main(String[] args) throws Exception{
        // TODO Auto-generated method stub
        MyServerStub stub=new MyServerStub();
        Add p=new Add();
        p.setA(11);
        p.setB(22);
        AddResponse res=stub.add(p);
        int c=res.get_return();
        System.out.println("Addition: "+c);
    }
}

```

Step 8 : Right click on **MyClient.java** file in Project Explorer. Click on **Run As -> Java Application**



You will get the output at the Console window as follows,



Troubleshooting :

We may get following error on running the above client

```
Exception in thread "main" org.apache.axis2.AxisFault:...
```

```
...
```

```
...
```

To overcome through this Exception, we need to make some changes in **services.xml** file. It can be located in the Web Services folder. To locate this **xml** file, follow the path as

Web Content-> WEB-INF-> Services->MyServer->META-INF. Locate services.xml

Change following lines

```
<messageReceivermep=http://www.w3.org/2004/08/wsdl/in-only
class="org.apache.axis2.rpc.receivers.RPCInOnlyMessageReceiver" />
<messageReceivermep=http://www.w3.org/2004/08/wsdl/in-out
class="org.apache.axis2.rpc.receivers.RPCMessageReceiver"/>
```

To

```
<messageReceivermep=http://www.w3.org/ns/wsdl/in-only
class="org.apache.axis2.rpc.receivers.RPCInOnlyMessageReceiver"/>
<messageReceivermep=http://www.w3.org/ns/wsdl/in-out
class="org.apache.axis2.rpc.receivers.RPCMessageReceiver"/>
```

Stop the Apache Tomcat Server and start it again. Now Run your **MyClient.java** file as Java Application and you will get the required output.

4.9 Describing Web Services : WSDL

- The WSDL is a Web Service Descriptor Language which is based on XML.
- The purpose of this file is to describe the web service and the method of accessing it.
- WSDL is a W3C recommendation.
- It contain various definitions. Following elements are described in WSDL file.

Elements	Description
types	It specifies the data types of the symbols used by the web service.
messages	It specifies the messages used by the web service.
portType	It specifies the name of the operation (function) defined by the web service
binding	It specifies the name of the protocol of the web service, typically it is SOAP.

1) Types

If we declare and use some variable in the web service then their data types can be specified.

For example -

```
<?xml version="1.0" encoding="UTF-8"?>
<definitions name="SimpleCalculator" targetNamespace="http://tempuri.org/wsdl" >
  <types>
    <schema targetNamespace="http://tempuri.org/types">
      <import namespace="http://schemas.xmlsoap.org/soap/encoding/" />
      <complexType name="CalciClass">
        <sequence>
          <element name="operator1" type="double"/>
          <element name="operator2" type="double"/>
          <element name="operation" type="String"/>
        </sequence>
      </complexType>
    </schema>
  </types>
```

2) Messages

The messages define the **part** of the element along with the data types. The message name specified the name of the function.

In following example the **HelloFun** is a name of the function which is declared in the interface named **MyInterface**. The return parameter is specified by **name="MyInterface_HelloFunResponse"**.

For example -

```
<?xml version="1.0" encoding="UTF-8"?>
<definitions name="SimpleHelloMessage" targetNamespace="http://tempuri.org/wsdl" >
<types/>
  <message name="MyInterface_HelloFun">
    <part name="String_1" type="xsd:string"/>
  </message>
  <message name="MyInterface_HelloFunResponse">
    <part name="result" type="xsd:string"/>
  </message>
```

3) PortTypes

This is an important element of the WSDL file. It defined various **operations** that can be performed by the web service.

Various operation types are -

Elements	Description
one-way	This operation will send the message but it will not get any response.
request-response	This operation will send a message and will receive a response.
Solicit-response	This operation will send a message and will wait for a response.
Notification	This operation will send a message and will not wait for a response.

The messages in portTypes define the name of the operation along with the prefix **tns** where tns stands for TypeNamespace.

4) Binding

The binding defines the message format and details of the protocol.

```
<?xml version="1.0" encoding="UTF-8"?>
<definitions name="SimpleHelloMessage" targetNamespace="http://tempuri.org/wsdl" >
<types/>
...
...

<binding name="MyInterfaceBinding" type="tns:MyInterface">
  <soap:binding transport="http://schemas.xmlsoap.org/soap/http" style="rpc"/>
  <operation name="HelloFun">
...
...
...

```

There are three main elements -

binding

The binding defines **name** which defines the name of the binding and **type** attributes specifies the port which is typically the name of the interface file.

soap:binding

The soap:binding defines **transport** and **style** attributes. The style is remote procedure call which allows the distributed clients to access the web service.

operation

The operations defines the port operations. Each operation contains the **soap:operation**, **input** and **output** elements. The **soap:body** element provides the communication details such as whether or not the associated message is encoded and if it is encoded then the **encodingStyle** is specified.

5) Service

The service element specifies the name for overall web service.

```
<?xml version="1.0" encoding="UTF-8"?>
<definitions name="SimpleHelloMessage" targetNamespace="http://tempuri.org/wsdl" >
  <types/>
  ...
  ...
  ...
  <service name="SimpleHelloMessage">
    <port name="MyInterfacePort" binding="tns:MyInterfaceBinding">
      <soap:address location="REPLACE_WITH_ACTUAL_URL"/>
    </port>
  </service>
</definitions>
```

4.10 Communicating Object Data : SOAP

- **Simple Object Access Protocol (SOAP)** is a protocol based on XML. It is used by the web services for exchange of information. It is a W3C recommendation.
- This exchange of information is done over HTTP. This is a platform and language independent protocol.
- The client-server communication is based on RPC. The HTTP does not designed to handle the distributed objects that are required by the RPC. Hence another application protocol is build over HTTP which popularly known as SOAP. SOAP allows to talk different applications that are running in two different operating systems.

4.10.1 Structure of SOAP

- The structure of the SOAP is defined by four building blocks.

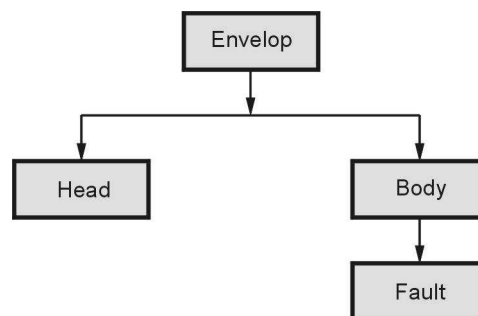


Fig. 4.10.1 SOAP building blocks

1) Envelop

The Envelop is the root of any SOAP document. It identifies that the XML document as a SOAP message. It defines two things - **Namespace** and **encodingStyle**. The Namespace value can be given as -

```
xmlns:soap="http://www.w3.org/2001/12/soap-envelope".
```

The encoding style is useful to define the data types used in the document. For example the value of encoding can be given as -

```
soap:encodingStyle="http://www.w3.org/2001/12/soap-encoding"
```

2) Header

The SOAP header field is optional. It contains the header information. The header contains three attributes such as **mustUnderstand**, **actor** and **encodingStyle**

The **mustUnderstand** attribute specifies 0 or 1. If the value is 1 then it means that the receiver should process the header.

Normally the SOAP message travels over a message path having the endpoints at the two ends. The SOAP **actor** is used to specify the URI for this endpoint.

The **encodingStyle** is useful to define the data types used in the document

The SOAP header defines how the recipient should process the SOAP message.

3) Body

The required SOAP Body element contains the actual SOAP message intended for the ultimate endpoint of the message.

4) Fault

This is optional element of the SOAP message. It is used to represent the error code. It consists of elements such as **faultcode** which is a code for fault. The **faultstring** message gives the details about the fault.

4.10.2 SOAP and HTTP

HTTP communicates over TCP/IP. HTTP is based on Request-Response mechanism. SOAP is a HTTP Request-Response protocol. SOAP is a combination of HTTP and XML.

The SOAP request can be HTTP GET or HTTP POST request. Each request consists of two HTTP headers : **content-type** and **content-length**. The content-type specifies the MIME type and the content-length specifies the size of the message in bytes. After this header the body part begins which defines SOAP Envelop, Header, Body and Fault.

Example

The SOAP request can be

```
POST /mycreation HTTP/1.0
Host: www.mywebsite.com
Content-Type: text/xml; charset=utf-8
Content-Length: 500
<?xml version="1.0"?>
<SOAP-ENV:Envelope
xmlns:SOAP-ENV="http://www.w3.org/2001/12/soap-envelope"
```

```

SOAP-ENV:encodingStyle="http://www.w3.org/2001/12/soap-encoding">
  <SOAP-ENV:Body xmlns:m="http://www.mywebsite.com/mycreation">
    <m:GetDetails>
      <m:Name>ABCD</m:Name>
    </m:GetDetails>
  </SOAP-ENV:Body>
</SOAP-ENV:Envelope>

```

The SOAP response can be

```

HTTP/1.0 200 OK
Content-Type: text/xml; charset=utf-8
Content-Length: 500
<?xml version="1.0"?>
<SOAP-ENV:Envelope
  xmlns:SOAP-ENV="http://www.w3.org/2001/12/soap-envelope"
  SOAP-ENV:encodingStyle="http://www.w3.org/2001/12/soap-encoding">
  <SOAP-ENV:Body xmlns:m="http://www.mywebsite.com/mycreation">
    <m:GetDetailsResponse>
      <m:ItemDetails>This is a my new creation</m:ItemDetails>
    </m:GetDetailResponse>
  </SOAP-ENV:Body>
</SOAP-ENV:Envelope>

```

4.10.3 SOAP Encoding

SOAP makes use of XML Schema to encode the programming constructs. Following are examples of SOAP encoding.

1. Encoding of Struct Data

Following is a code fragment which can be encoded as follows -

```

<xs:element name="Student">
  <xs:complexType>
    <xs:sequence>
      <xs:element name="Name" type="xs:string"/>
      <xs:element name="RollNo" type="xs:integer"/>
      <xs:element name="address" type="tns:Address"/>
    </xs:sequence>
    <xs:element name="Address">
      <xs:complexType>
        <xs:sequence>
          <xs:element name="street" type="xsd:string"/>
          <xs:element name="city" type="xsd:string"/>
          <xs:element name="country" type="xsd:string"/>
        </xs:sequence>
      </xs:complexType>
    </xs:element>
  </xs:complexType>
</xs:element>

```

The following is an example of how the preceding schema definition could be subsequently used in a SOAP encoding.

```
<e:Student>
  <Name>AAA</Name>
  <RollNo>11</RollNo>
  <address>
    <street>XYZ Road</street>
    <city>PQR City</city>
    <country>India</country>
  </address>
</e:Student>
```

2. Encoding of Arrays

An array in SOAP is of type **SOAP-ENC:Array**

```
<element name="myArray"
  type="SOAP-ENC:Array"/>
```

The preceding schema can be represented in SOAP encoding as follows -

```
<myArray SOAP-ENC:arrayType="xsd:int[3]">
  <number>1</number>
  <number>2</number>
  <number>3</number>
</myArray>
```

The above encoding shows that there is an array of three integers. All these members of array are called **number**.

4.10.4 RPC Representation

- The RPC (Remote Procedure Call) is a mechanism used when the client makes a call to a method that resides on another machine. The SOAP has a support for RPC representation.
- The **style** attribute of **soap:binding** element in the WSDL document makes use of string "rpc" to denote that the communication with the client is in RPC style.

```
<soap:binding transport="http://schemas.xmlsoap.org/soap/http"
  style="rpc"/>
```

- The **soap:body** element indicated using an encoding specified by SOAP 1.1 to represent RPC.
- In SOAP specified representation of RPC calls, the call is modeled as **struct**.
- The specified structure used in SOAP representation of RPC call has a name for the remote procedure call and has one accessor for each parameter that is to be passed to the procedure. This struct is then encoded as XML markup. This markup is then made a child of SOAP body element.

For example - Following is a soap document sent by the client to display message web service.

```
<?xml version="1.0" encoding="UTF-8"?>
<env:Envelop
  xmlns:env="http://schemas.xmlsoap.org/soap/envelop/"
  xmlns:xsd="http://www.w3.org/2001/XMLSchema"
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xmlns:enc="http://schemas.xmlsoap.org/soap/encoding/"
  xmlns:ns0="http://tempuri.org/wsdl"
  xmlns:ns1="http://tempuri.org/types"
  env:encodingStyle="http://schemas.xmlsoap.org/soap/encoding/">
  <env:Body>
    <ns0:HelloMsg>          ← Call to procedure which is RPC
      <msg_1 xsi:type=:xsd:string>Welcome</msg_1>
    </ns0:HelloMsg>
  </env:Body>
</env:Envelop>
```

Part III : Struts

4.11 Overview

SPPU : Dec.-18, Marks 4

- Struts is a framework software used for developing the Java Web Applications.
- The struts framework was initially created by Craig McClanahan and donated to Apache Software Foundation.
- It is an open source product.
- The latest commonly used release is **struts 2**.

Features

1. It is based on Model view Controller (MVC) architecture.
2. The action class is POJO (Plain Old Java Object) that means it is a simple Java class.
3. Struts2 provides the support to AJAX technology. That means only required field can be sent to the server and no need to send entire page. Due to this feature the execution is fast.
4. The struts2 framework can be integrated with other applications like Hibernate, Spring and so on.
5. Struts2 provides tag support such as UI tags, Data tags, control tags etc.
6. It provides support for views and templates.
7. Struts2 support multiple views for the same application.

Advantages of Struts

1. The complex web applications can be built using simple Model, View and Controller components.
2. The designs made using struts are simpler, consistent and easy to maintain.

Review Question

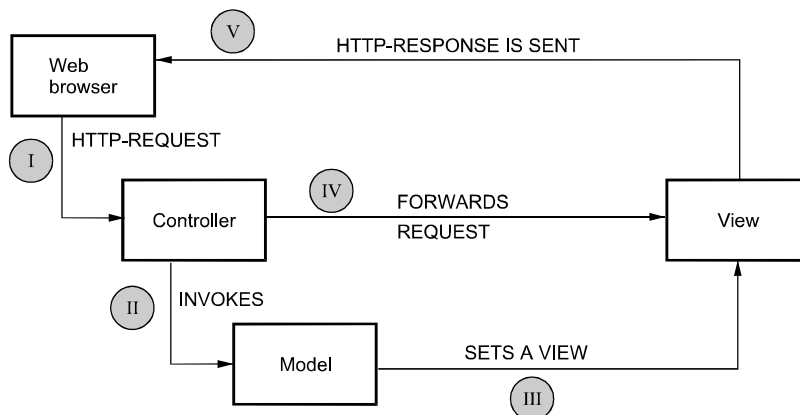
1. Write various features of struts framework.

SPPU : Dec.-18, Marks 4

4.12 Architecture

SPPU : May-18,19, Marks 6

- The struts framework is based on **Model, View and Controller** architecture (MVC).
 - I. Normally when client wants any web page he demands it using get or post request or by clicking Submit button, then a controller is invoked.
 - II. A Controller is nothing but a Java class in which **business logic** is written. The job of Controller is to take user input and pass it to the **Model**. The controller in turn invokes the **Model**.
 - III. Model stores the user's data may be in databases or so, and returns some result to the Controller.
 - IV. Using this result, the Controller figures out user requirements and accordingly forwards the result to the View page.
 - V. Finally required view can be obtained as a response (i.e. a display of required web page is obtained) on the web browser. This can be diagrammatically shown as below -

**Fig. 4.12.1**

In struts,

- **Model** components provide a model for "business logic" and data behind struts program.
- **View** components in struts are those components that present the information to the user or accept the user input. Typically these components are JSP or HTML pages. View components are responsible for providing the display of required information. Struts provide large number of JSP custom tags or **struts tags** to simplify the development of view components.
- **Controllers** are the Java classes which allow developers to remove error handling activities from JSP pages. These classes are typically referred as **action classes**.

Review Question

1. Draw and explain neat diagram which depicts MVC to the struts architecture.

SPPU : May-18,19, Marks 6

4.13 Execution of First Struts Applications

Environment : Before execution of any struts programs following software must be installed.

1) JDK 2) Apache Tomcat 3) Eclipse IDE

4) **Struts 2 library :** For installing struts 2 libraries get them downloaded from

<https://struts.apache.org/download.cgi>

I have downloaded **struts-2.2.34-all.zip** and on unzipping it will give directory structure **c:\struts-2.2.34**

Step 1 : Start Eclipse. From **File Menu** select **File -> New->Dynamic Web Project**. Type the name of the project. In my case I have created the project by the name **FirstStrutsDemo**. The screen-shot will be Just click next button with default options and check the **Generate Web.xml deployment descriptor** option. This will create a dynamic web project in Eclipse.

The screenshot shows the 'New Project' dialog in the Eclipse IDE. The 'Project name' field is filled with 'FirstStrutsDemo'. The 'Project location' section has the 'Use default location' checkbox checked, and the 'Location' field shows 'D:\StrutsProjects\FirstStrutsDem'. The 'Target runtime' is set to 'Apache Tomcat v7.0'. The 'Dynamic web module version' is set to '3.0'. The 'Configuration' section shows 'Default Configuration for Apache Tomcat v7.0'. The 'EAR membership' section has the 'Add project to an EAR' checkbox unchecked. The 'Working sets' section has the 'Add project to working sets' checkbox unchecked. Buttons for 'Browse...', 'New Runtime...', 'Modify...', 'New Project...', and 'Select...' are visible next to their respective fields.

Project name: FirstStrutsDemo

Project location

☒ Use default location

Location: D:\StrutsProjects\FirstStrutsDem Browse...

Target runtime

Apache Tomcat v7.0 New Runtime...

Dynamic web module version

3.0

Configuration

Default Configuration for Apache Tomcat v7.0 Modify...

A good starting point for working with Apache Tomcat v7.0 runtime. Additional facets can later be installed to add new functionality to the project.

EAR membership

☐ Add project to an EAR

EAR project name: EAR New Project...

Working sets

☐ Add project to working sets

Working sets: Select...

Step 2 : Now copy the Jar files present in the **c:\struts-2.2.34\lib** folder to our project's WEB-INF\lib folder. Following files need to be copied

- commons-fileupload-1.3.2
- commons-io-2.2
- commons-lang-2.4
- commons-lang3-3.2
- commons-logging-1.1.3

freemarker-2.3.22
javassist-3.11.0.GA
ognl-3.0.21
struts2-core-2.3.34
xwork-core-2.3.34

Step 3 : Now create an action class which is a Java class. It should be created under **Java-Resources->src** folder. Under **src** folder create a package with suitable name. I have created package named **myPkg** and Java source class as **MyBigHello.java**. The code for this is as follows -

MyBigHello.java

```
package myPkg;  
public class MyBigHello {  
    private String name;  
    public String execute() throws Exception {  
        return "success";  
    }  
    public String getName() {  
        return name;  
    }  
    public void setName(String name) {  
        this.name = name;  
    }  
}
```

The action class gets executed when user clicks the URL. Based on it, the JSP page will be displayed.

Step 4 : Now we will create a **view** page. I have named it as **MyHello.jsp**. in the **WebContent** folder in your eclipse project. To do this, right click on the **WebContent** folder in the project explorer and select New >JSP File.

MyHello.jsp

```
<%@ page contentType = "text/html; charset = UTF-8" %>  
<%@ taglib prefix = "s" uri = "/struts-tags" %>  
<html>  
    <head>  
        <title>My First Struts Demo Application</title>  
    </head>  
    <body>  
        <h3>Welcome my friend, <s:property value = "name"/></h3>  
    </body>  
</html>
```

In above code, the taglib directive tells us the Servlet container that this page is using Struts 2 tag and the tags will be preceded by s.

The **name** property define in **MyBigHello.java** class will be returned by the function **getName()** of **MyBigHello** class.

Step 5 : Now we will create the main page. In struts 2 the main page is denoted by **index.jsp** in the WebContent folder. This file serves as the initial action URL and calls the method defined in **MyBigHello** class, accordingly the display of **MyHello.jsp** page will be updated.

index.jsp

```
<%@ page language = "java" contentType = "text/html; charset = ISO-8859-1"
    pageEncoding = "ISO-8859-1"%>
<%@ taglib prefix = "s" uri = "/struts-tags"%>
<!DOCTYPE html PUBLIC "-//W3C//DTD HTML 4.01 Transitional//EN"
"http://www.w3.org/TR/html4/loose.dtd">
<html>
<head>
<title>My First Struts Demo Application</title>
</head>
<body>
<h2>My Struts2 Application</h2>
<form action = "hello">
<label for = "name">Enter your name</label> <br/>
<input type = "text" name = "name"/>
<input type = "submit" value = "Submit"/>
</form>
</body>
</html>
```

The **hello** action defined in the above view file will be mapped to the **MyBigHello** class and its execute method using **struts.xml** file. When a user clicks on the Submit button it will cause the Struts 2 framework to run the execute method defined in the **MyBigHello** class and based on the returned value of the method, an appropriate view will be selected and rendered as a response.

Step 6 : There is a file that ties **MyBigHello class(model)** with **MyHello(view)**. This file is **struts.xml**. This file must be present in the classes folder. Hence under WebContent/WEB-INF, create a folder named classes and add new file named struts.xml. The code for this file is

struts.xml

```
<?xml version = "1.0" encoding = "UTF-8"?>
<!DOCTYPE struts PUBLIC
"-//Apache Software Foundation//DTD Struts Configuration 2.0//EN"
"http://struts.apache.org/dtds/struts-2.0.dtd">
<struts>
  <constant name = "struts.devMode" value = "true" />
  <package name="default" extends="struts-default">
    <action name = "hello"
      class = "myPkg.MyBigHello"
      method = "execute">
      <result name = "success">/MyHello.jsp</result>
    </action>
  </package>
</struts>
```

Step 7 : Now create web.xml i.e. deployment descriptor file. This file acts as an entry point for the request made by struts 2. The entry point will be **filer defined**. Hence, we will define an entry of org.apache.struts2.dispatcher.FilterDispatcher class in web.xml. The web.xml file needs to be created under the WEB-INF folder under WebContent. The **web.xml** generated by Eclipse can be modified as -

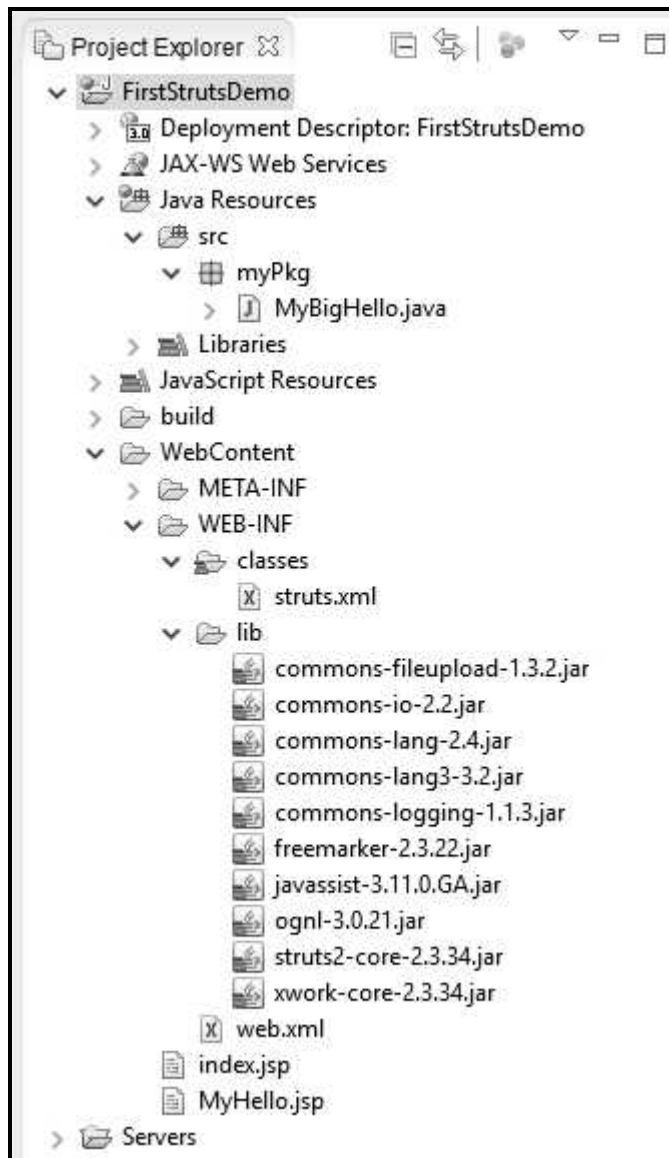
web.xml

```
<?xml version = "1.0" encoding = "UTF-8"?>
<web-app xmlns:xsi = "http://www.w3.org/2001/XMLSchema-instance"
  xmlns = "http://java.sun.com/xml/ns/javaee"
  xmlns:web = "http://java.sun.com/xml/ns/javaee/web-app_2_5.xsd"
  xsi:schemaLocation = "http://java.sun.com/xml/ns/javaee
  http://java.sun.com/xml/ns/javaee/web-app_3_0.xsd"
  id = "WebApp_ID" version = "3.0">
  <display-name>Struts 2</display-name>
  <welcome-file-list>
    <welcome-file>index.jsp</welcome-file>
  </welcome-file-list>

  <filter>
    <filter-name>struts2</filter-name>
    <filter-class>
      org.apache.struts2.dispatcher.FilterDispatcher
    </filter-class>
  </filter>

  <filter-mapping>
    <filter-name>struts2</filter-name>
    <url-pattern>/*</url-pattern>
  </filter-mapping>
</web-app>
```

Following screen-shot of Project-Explorer represents the location of above created files.

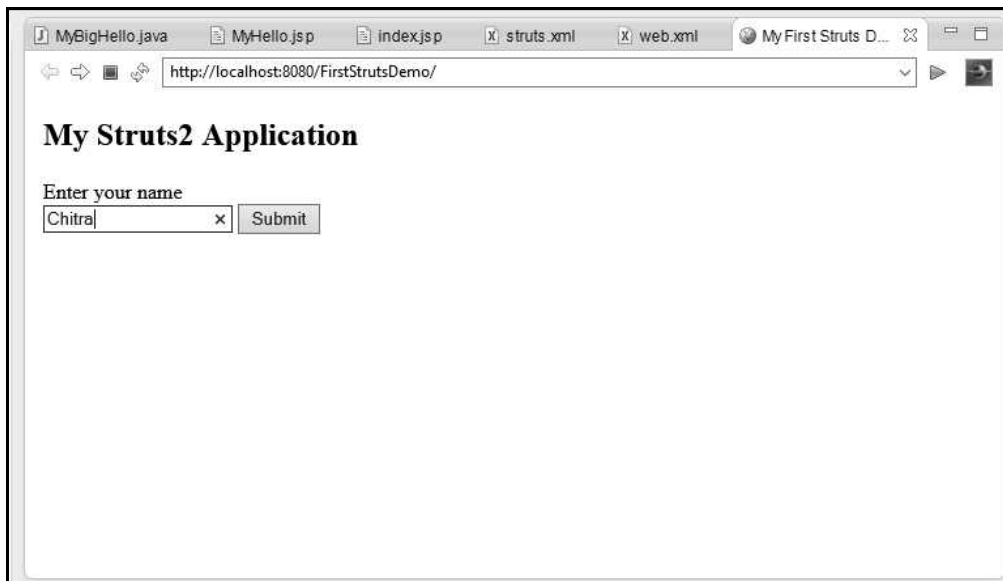


Step 8 : Execution of application :

- i) For executing the application right click on the project name and click **Export > WAR File** to create a War file. Then deploy this WAR in the Tomcat's webapps directory. For a **destination** click the **Browse** button and select the path `c:/your_tomcat_directory/webapps` folder by accepting the default name of the war file. Normally "your_project_name.war" file gets generated. Here we get **FirstStrutsDemo.war** in the `c:/tomcat_directory/webapps`.

- ii) Now run the Tomcat Web server. By selecting the **Run configurations...** from the Menu bar of Eclipse. Select the Apache, under which select the Tomcat server which is same as the Tomcat server which you have already installed on your computer. I have selected Tomcat v7.0 as the version of my already installed Tomcat. Then as a destination select the path of your tomcat directory. After finishing this the Tomcat web server gets started.

The output can be displayed on the web browser as



Click Submit button and we will get



4.14 Configuration

SPPU : Dec.-18,19, Marks 6

There are two main configuration files used in struts and those are **struts.xml** and **web.xml**.

There is **struts.properties** file is used to override the default values of default.xml file provided by struts framework. So it is not mandatory.

struts.xml

The struts.xml file is used to initialize the resources such as interceptors, Actions classes. This file is present inside the **WEB-INF/classes** folder.

The structure of this file is

```
<?xml version = "1.0" Encoding = "UTF-8"?>
<!DOCTYPE struts PUBLIC
  "-//Apache Software Foundation//DTD Struts Configuration 2.0//EN"
  "http://struts.apache.org/dtds/struts-2.0.dtd">
<struts>
  <constant name = "struts.devMode" value = "true" />
  <package name = "FirstStrutDemo" extends = "struts-default">

    <action name = "demo"
      class = "myPkg.TestAction"
      method = "execute">
      <result name = "success">/MyHello.jsp</result>
    </action>

    <!-- more actions can be listed here -->

  </package>
  <!-- more packages can be listed here -->
</struts>
```

1) Package Element

The package element specifies module or your project name. It has following attributes -

- i) **name** : The name must define the package
- ii) **namespace** : This is optional attribute. If namespace is not present, / is assumed as the default namespace.
- iii) **extends** : The package element mostly extends the struts-default package where interceptors and result types are defined.

2) Action Element

- This element represents action. Various attributes of this element are -
 - i) **name** : This is any name with which the action is associated.
 - ii) **class** : The name of the action class is specified. This name must be accompanied with the package name inside which the action class is defined.
 - iii) **method** : This is optional field. By default the **execute** method is associated with this attribute.
 - iv) **result name** : The result name can be success, error and so on. In struts, on result element the page to be displayed on success or error is specified.

web.xml

The web.xml is a configuration file which determines how elements of the HTTP request are processed by the servlet container. It is not strictly a Struts2 configuration file, but it is a file that needs to be configured for Struts2 to work.

The web.xml file needs to be created under the folder **WebContent/WEB-INF**.

This file contains the <welcome-file-list> tag using which the first file to be loaded on execution of struts project is specified. For example

```
<welcome-file-list>
  <welcome-file>index.jsp</welcome-file>
</welcome-file-list>
```

The simple meaning of above code is that as soon as you execute your struts application, load the **index.jsp** file.

```
<filter-mapping>
  <filter-name>struts2</filter-name>
  <url-pattern>/*</url-pattern>
</filter-mapping>
```

The above code indicates that <url-pattern> is /* means that all urls will be parsed by the struts filter.

Review Question

1. What are the different configuration files are require to develop any struts application ? Explain each configuration file.

SPPU : Dec.-18, 19, Marks 6

4.15 Actions

SPPU : Dec.-18, May-19, Marks 6

- Actions are important element of struts application. The actions are basically the Java programs that contain the processing logic and satisfy the requests made by the user.

- Each URL is mapped to these actions.
- Action performs two tasks -
 - Action plays an important role from request to view. This view is written in JSP.
 - Action directs the framework to display particular view as an response to the request.
- Action class is a POJO (Plain Old Java Object) class which has attributes and methods. There must be one method that should always be present in action class and that is - **execute()** method.
- By default **execute()** method is called when client request goes to action class.
- This class is bean class. That means it is a Java class containing **getXX** and **setXX** methods.
- To understand the **getXX** and **setXX** methods - consider that there is a Java class named Test inside which there are two attributes namely -name and roll. Then in this case **getName()**, **setName**, **getRoll()** and **setRoll()** methods will be created and defined. The sample code for such **Test** bean class would be

```
Public class Test
```

```
{  
private String name;  
private int roll;  
public String getName() {  
    return name;  
}  
public void setName(String name) {  
    this.name = name;  
}  
public String getRoll() {  
    return roll;  
}  
public void setRoll(int roll) {  
    this.roll = roll;  
}  
}
```

Note that the attribute name must appear after get or set and the first letter of the method name must be capital

We can write simple **Java class** as action class or extend that class for **ActionSupport** class. This **ActionSupport** class implements an important interface named **Action**. The **Action** interface is as follows -

```
public interface Action {  
    public static final String SUCCESS = "success";  
    public static final String NONE = "none";  
    public static final String ERROR = "error";  
    public static final String INPUT = "input";  
    public static final String LOGIN = "login";  
}
```

```
public String execute() throws Exception;  
}
```

How to implement Action class Using struts ?

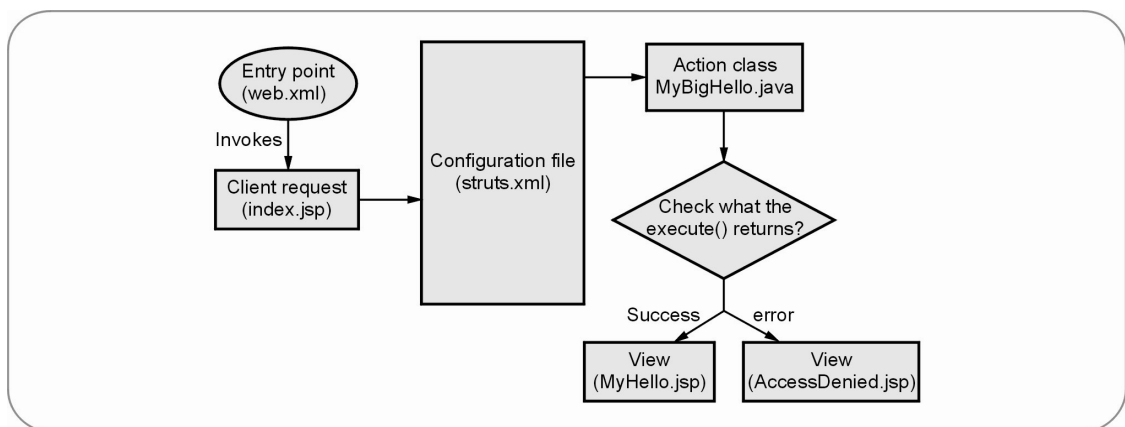
Step 1 : Start Eclipse. From File Menu select **File -> New->Dynamic Web Project**.

Type the name of the project.

Just click **next** button with default options and check the **Generate Web.xml deployment descriptor** option. This will create a dynamic web project in Eclipse

Step 2 : Now copy the required Jar files present in the **c:\struts-2.2.34\lib** folder to our project's **WEB-INF\lib** folder.

Before actually building the struts application just refer following figure. Because in this application, we are going to create multiple files. The relationship and purpose of these files is depicted by following figure -



Step 3 : Now create an action class which is a Java class. It should be created under **Java-Resources->src** folder. Under **src** folder create a package with suitable name. I have created package named **myPkg** and Java source class as **MyBigHello.java**. The code for this is as follows -

MyBigHello.java

```
package myPkg;  
public class MyBigHello {  
    private String name;  
    public String execute() throws Exception {  
        return "success";  
    }  
}
```

```
public String getName() {  
    return name;  
}  
public void setName(String name) {  
    this.name = name;  
}  
}
```

The action class gets executed when user clicks the URL. Based on it, the JSP page will be displayed.

Step 4 : Now we will create the main page. In struts 2 the main page is denoted by **index.jsp** in the **WebContent** folder. This file serves as the initial action URL and calls the method defined in **MyBigHello** class(We have defined this class in pervious step)

index.jsp

```
<%@ page language = "java" contentType = "text/html; charset = ISO-8859-1"  
    pageEncoding = "ISO-8859-1"%>  
<%@ taglib prefix = "s" uri = "/struts-tags"%>  
    <!DOCTYPE html PUBLIC "-//W3C//DTD HTML 4.01 Transitional//EN"  
    "http://www.w3.org/TR/html4/loose.dtd">  
<html>  
    <head>  
        <title>My First Struts Demo Application</title>  
    </head>  
    <body>  
        <h2>My Struts2 Application</h2>  
        <form action = "hello">  
            <label for = "name">Enter your name</label> <br/>  
            <input type = "text" name = "name"/>  
            <input type = "submit" value = "Submit"/>  
        </form>  
    </body>  
</html>
```

The **hello** action defined in the above view file will be mapped to the **MyBigHello** class and it executes method using **struts.xml** file. When a user clicks on the Submit button it will cause the Struts 2 framework to run the execute method defined in the **MyBigHello** class and based on the returned value of the method, an appropriate view will be selected and rendered as a response.

Step 5 : Create a file named **struts.xml**. This file must be present in the **classes** folder. Hence under WebContent/WEB-INF, create a folder named **classes** and add new file named **struts.xml**. The code for this file is

struts.xml

```
<?xml version = "1.0" encoding = "UTF-8"?>
<!DOCTYPE struts PUBLIC
  "-//Apache Software Foundation//DTD Struts Configuration 2.0//EN"
  "http://struts.apache.org/dtds/struts-2.0.dtd">
<struts>
  <constant name = "struts.devMode" value = "false" />
  <package name="default" extends="struts-default">
    <action name = "hello"
      class = "myPkg.MyBigHello"
      method = "execute">
      <result name = "success">/MyHello.jsp</result>
      <result name = "error">/AccessDenied.jsp</result>
    </action>
  </package>
</struts>
```

The **struts.xml** is a file that ties **MyBigHello class(model)** with **MyHello(view)**. This **MyHello.jsp** file is basically a view file. It is created in next step.

Step 6 : We will create a **view** page. I have named it as **MyHello.jsp**. in the **WebContent** folder in your eclipse project. To do this, right click on the **WebContent** folder in the project explorer and select New >JSP File.

MyHello.jsp

```
<%@ page contentType = "text/html; charset = UTF-8" %>
<%@ taglib prefix = "s" uri = "/struts-tags" %>
<html>
  <head>
    <title>My First Struts Demo Application</title>
  </head>
  <body>
    <h3>Welcome my friend, <s:property value = "name"/></h3>
  </body>
</html>
```

The **name** property define in **MyBigHello.java** class will be returned by the function **getName()** of **MyBigHello** class.

Step 7 : If we do not enter the name as "Admin" then the error page will be displayed as

AccessDenied.jsp

```
<%@ page contentType = "text/html; charset = UTF-8" %>
<%@ taglib prefix = "s" uri = "/struts-tags" %>
<html>
  <head>
```

```
<title>Access Denied</title>
</head>
<body>
  Error!!!You are not allowed to view this page
</body>
</html>
```

Step 8 : Now create web.xml i.e. deployment descriptor file. This file acts as an entry point for the request made by struts 2. The entry point will be **filer defined**. Hence, we will define an entry of **org.apache.struts2.dispatcher.FilterDispatcher** class in **web.xml**. The **web.xml** file needs to be created under the WEB-INF folder under WebContent. The **web.xml** generated by Eclipse can be modified as -

web.xml

```
<?xml version = "1.0" encoding = "UTF-8"?>
<web-app xmlns:xsi = "http://www.w3.org/2001/XMLSchema-instance"
  xmlns = "http://java.sun.com/xml/ns/javaee"
  xmlns:web = "http://java.sun.com/xml/ns/javaee/web-app_2_5.xsd"
  xsi:schemaLocation = "http://java.sun.com/xml/ns/javaee
  http://java.sun.com/xml/ns/javaee/web-app_3_0.xsd"
  id = "WebApp_ID" version = "3.0">
  <display-name>Struts 2</display-name>
  <welcome-file-list>
    <welcome-file>index.jsp</welcome-file>
  </welcome-file-list>

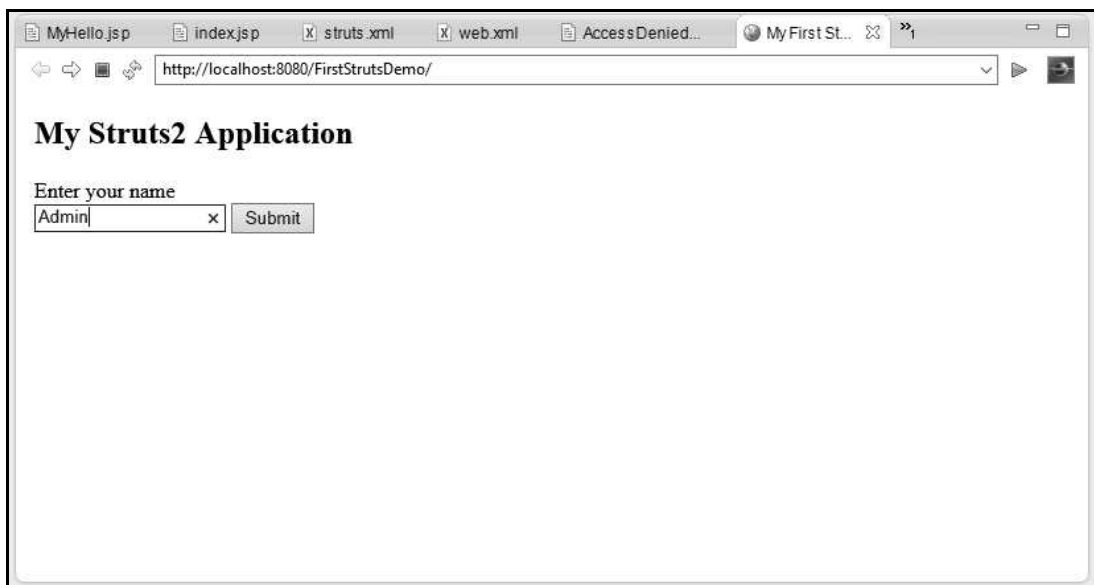
  <filter>
    <filter-name>struts2</filter-name>
    <filter-class>
      org.apache.struts2.dispatcher.FilterDispatcher
    </filter-class>
  </filter>
  <filter-mapping>
    <filter-name>struts2</filter-name>
    <url-pattern>/*</url-pattern>
  </filter-mapping>
</web-app>
```

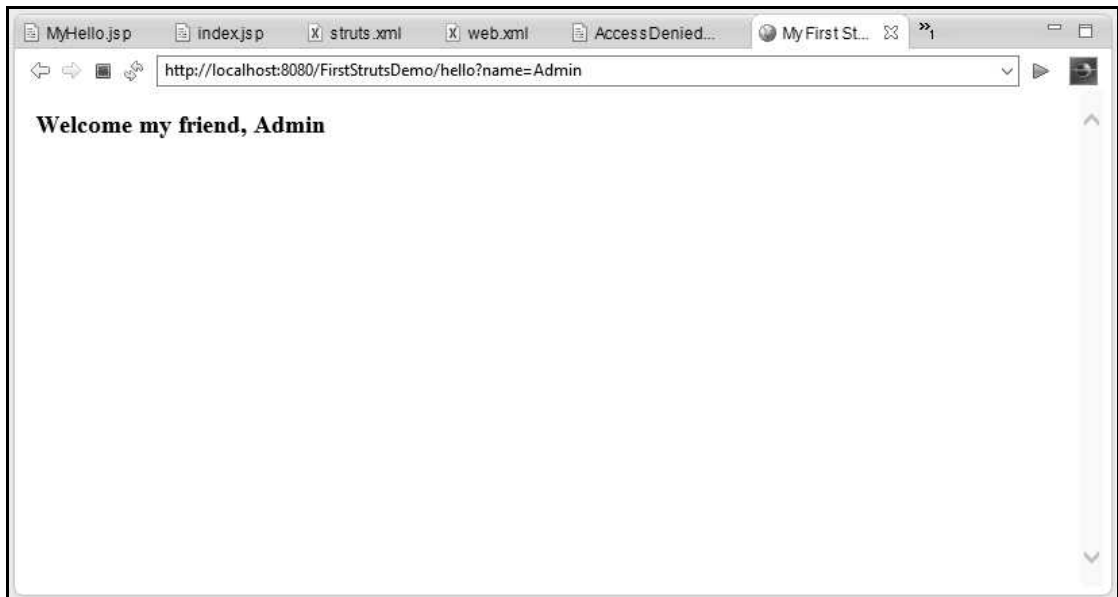
For understanding the location of above files I am giving the following screen-shot. Note that these files must be created in respective folders only -



Step 9 : Now execute the application.

The output will be





If we enter other than Admin, here I have given the name as 'Parth' and then -



Review Question

1. Identify and explain data tags from struts2 which are used to manipulate data displayed on any web application page.

SPPU : Dec.-18, May-19, Marks 6

4.16 Interceptors**SPPU : May-18, Marks 6**

- Interceptor is an object which intercepts an action dynamically. It executed before and after the action execution.
- In Struts 2, interceptor is used to perform operations such as validation, exception handling, internationalization, displaying intermediate result etc.
- Following is a list of Inceptors that are preconfigured and can be ready to use.
 - 1) **alias** : It converts similar parameters that have different names between requests.
 - 2) **chain** : If it is used with chain result type, it makes the properties of previous action available in the current action.
 - 3) **checkbox** : It is used to handle the check boxes in the form. By this, we can detect the unchecked checkboxes.
 - 4) **cookie** : It adds a cookie to the current action.
 - 5) **conversionError** : It adds conversion errors to the action's field errors.
 - 6) **createSession** : It creates and HttpSession object if it doesn't exists.
 - 7) **clearSession** : It unbinds the HttpSession object.
 - 8) **debugging** : It provides support of debugging.
 - 9) **execAndWait** : It sends an intermediate waiting page for the result.
 - 10) **exception** : It maps exception to a result.
 - 11) **fileUpload** : It provides support to file upload in struts 2.
 - 12) **i18n** : It provides support to internationalization and localization.
 - 13) **store** : It stores and retrieves action messages, action errors or field errors for action that implements ValidationAware interface.
 - 14) **params** : It populates the action properties with the request parameters.
 - 15) **scope** : It is used to store the action state in the session or application scope.
 - 16) **timer** : It outputs the time needed to execute an action.
 - 17) **validation** : It provides support to input validation.
- We can create our own Inceptor object. This custom based inceptor can be plugged in struts.
- Framework creates an object of **ActionInvocation** that encapsulates the action and all the interceptors configured for that action.

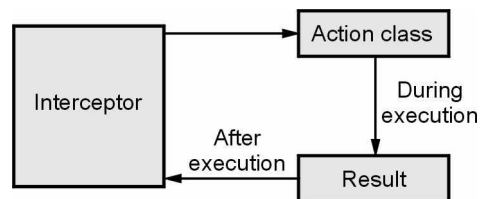
- Each interceptors are called before the action gets called.
- Once the action is called and result is generated, each interceptors are again called in reverse order to perform post processing work.
- Interceptors can alter the workflow of action. It may prevent the execution of action.

Advantages of Interceptors

1. If we want pre-processing services and post-processing services for an **Action** then these services can be provided by interceptor.
2. It provides pluggable functionalities for validation, exception handling and logging. We do not need to redeploy the application.
3. Interceptors can be coupled with request and response cycle.

How to Create Custom Intercept ?

- For creating the custom intercept we need interface **Interceptor** This interface contains a method **intercept()** which ultimately calls the **invoke()** method.
- The interceptor can be executed before calling the action, during execution of action and after execution of action.
- Let us create an interceptor in struts framework.



Step 1 : Start Eclipse. From File Menu select **File -> New->Dynamic Web Project**.

Type the name of the project.

Just click **next** button with default options and check the **Generate Web.xml deployment descriptor** option. This will create a dynamic web project in Eclipse

Step 2 : Now copy the required Jar files present in the **c:\struts-2.2.34\lib** folder to our project's **WEB-INF\lib** folder

Step 3 : Inside the **Java Resources-> src** folder create a package(I have created **myPkg** package). Inside that package create a Java class for inceptor named **MyInterceptor** as follows -

MyInterceptor.java

```
package myPkg;
import com.opensymphony.xwork2.ActionInvocation;
import com.opensymphony.xwork2.interceptor.AbstractInterceptor;
public class MyInterceptor extends AbstractInterceptor{
    public String intercept(ActionInvocation invocation)throws Exception {
        System.out.println("Before Execution");
        String result = invocation.invoke();
        System.out.println("After Execution");
        return result;
    }
}
```

Remember these interceptors will shows the result in the console automatically

Step 4 : Now create the Java class as action class in which execute() method is defined. Again this class is created inside the user defined package of src folder. The code for this is as follows -

Test.java

```
package myPkg;
import com.opensymphony.xwork2.ActionSupport;
public class Test extends ActionSupport{
    private String name;
    public String execute() throws Exception {
        System.out.println("Inside execute() method!!!");
        return "success";
    }
    public String getName() {
        return name;
    }
    public void setName(String name) {
        this.name = name;
    }
}
```

Step 5 : Now we will create a **view** page. I have named it as **MyHello.jsp** in the **WebContent** folder in your eclipse project. To do this, right click on the **WebContent** folder in the project explorer and select **New >JSP File**.

MyHello.jsp

```
<%@ page contentType = "text/html; charset = UTF-8" %>
<%@ taglib prefix = "s" uri = "/struts-tags" %>
<html>
<head>
```

```
<title>My Interceptor Demo Application</title>
</head>
<body>
  <h3>Welcome my friend, <s:property value = "name"/></h3>
</body>
</html>
```

Step 6 : Now create the main page as **index.jsp** in the **WebContent** folder.

index.jsp

```
<%@ page language = "java" contentType = "text/html; charset = ISO-8859-1"
    pageEncoding = "ISO-8859-1"%>
<%@ taglib prefix = "s" uri = "/struts-tags"%>
  <!DOCTYPE html PUBLIC "-//W3C//DTD HTML 4.01 Transitional//EN"
"http://www.w3.org/TR/html4/loose.dtd">
<html>
  <head>
    <title>Struts Inceptor Demo Application</title>
  </head>
  <body>
    <h2>My Struts2 Application</h2>
    <form action = "hello">
      <label for = "name">Enter your name</label> <br/>
      <input type = "text" name = "name"/>
      <input type = "submit" value = "Submit"/>
    </form>
  </body>
</html>
```

Step 7 : Create the configuration file **struts.xml** as follows-

struts.xml

```
<?xml version = "1.0" encoding = "UTF-8"?>
<!DOCTYPE struts PUBLIC
  "-//Apache Software Foundation//DTD Struts Configuration 2.0//EN"
  "http://struts.apache.org/dtds/struts-2.0.dtd">
<struts>
  <constant name = "struts.devMode" value = "true" />
  <package name="default" extends="struts-default">
    <interceptors>
      <interceptor name = "myinterceptor"
```

```

        class = "myPkg.MyInterceptor" />
    </interceptors>
    <action name = "hello"
        class = "myPkg.Test"
        method = "execute">
        <interceptor-ref name = "params"/>
        <interceptor-ref name = "myinterceptor" />
        <result name = "success">/MyHello.jsp</result>
    </action>
</package>
</struts>

```

Step 8 : The configuration file **web.xml** will invoke the **index.jsp** file. It is present inside WEB-INF folder. Just modify the web.xml file as follows -

web.xml

```

<?xml version="1.0" encoding="UTF-8"?>
<web-app xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xmlns="http://java.sun.com/xml/ns/javaee"
xmlns:web="http://java.sun.com/xml/ns/javaee/web-app_2_5.xsd"
xsi:schemaLocation="http://java.sun.com/xml/ns/javaee
http://java.sun.com/xml/ns/javaee/web-app_3_0.xsd" id="WebApp_ID" version="3.0">
    <display-name>InceptorDemo</display-name>
    <welcome-file-list>
        <welcome-file>index.jsp</welcome-file>
    </welcome-file-list>
    <filter>
        <filter-name>struts2</filter-name>
        <filter-class>
            org.apache.struts2.dispatcher.FilterDispatcher
        </filter-class>
    </filter>

    <filter-mapping>
        <filter-name>struts2</filter-name>
        <url-pattern>/*</url-pattern>
    </filter-mapping>
</web-app>

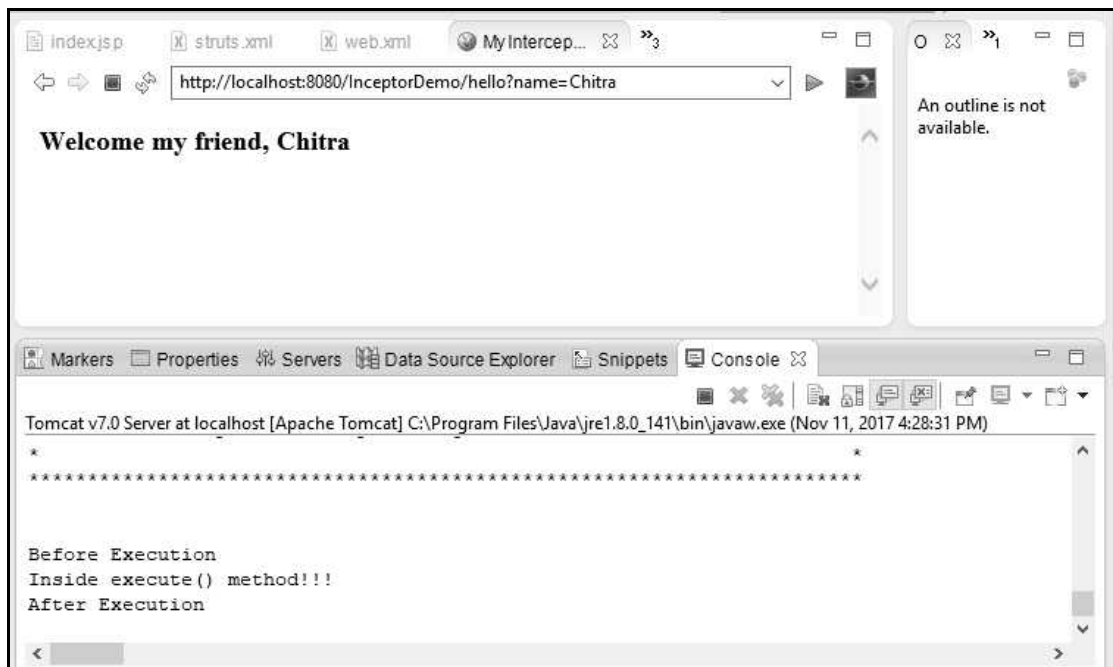
```

While creating this application in struts 2, please refer the following screen-shot of Project Explorer to know the location of the above created files.



Step 9 : Export the WAR file and start Tomcat server. Run the program on Tomcat server. The output will be obtained as follows -

Enter some name inside the text box and click the Submit button. You will get the execution of interceptor as follows :



Review Question

1. How to use interceptors in Struts 2 ? List and describe important interceptors provided by Struts 2 framework.

SPPU : May-18, Marks 6**4.17 Result Types**

- The result type is used to change the behaviour of view page to render the contents of action class.
- After executing the Action class, the target view page will be executed based on configuration in struts.xml.
- The Struts2 framework provides set of Result types like; chaining, dispatcher, redirect action, velocity and freemarket etc., that has to be configure in struts.xml for different kind of result.
- The `<result>` tag is used in **struts.xml** to render the view page. This result tag is provided with predefined result types such as dispatcher, redirect and so on. For example

```
<result name = "success" type="redirect">
```

- The **dispatcher** result type is the default type, and is used if no other result type is specified. It's used to forward to a servlet, JSP, HTML page, and so on, on the server. It can be written as

```
<result name="success" type="dispatcher">  
<param name="location">/Success.jsp</param>  
</result>
```

Let us understand how to use result type with demonstration.

How to use result type ?

In this application we will use result type for redirecting the view page to a new JSP page. Just follow the simple steps given below -

Step 1 : Create the files MyBigHello.java, index.jsp, MyHello.jsp and web.xml as it is discussed in section 4.16.

Step 2 : The struts.xml file is little bit changed. It contains the **<result ... type>** parameter. Following file tells us that on success redirect to a new page. This new page is **NewLocation.jsp**

struts.xml

```
<?xml version = "1.0" encoding = "UTF-8"?>
<!DOCTYPE struts PUBLIC
  "-//Apache Software Foundation//DTD Struts Configuration 2.0//EN"
  "http://struts.apache.org/dtds/struts-2.0.dtd">

<struts>
  <constant name = "struts.devMode" value = "false" />

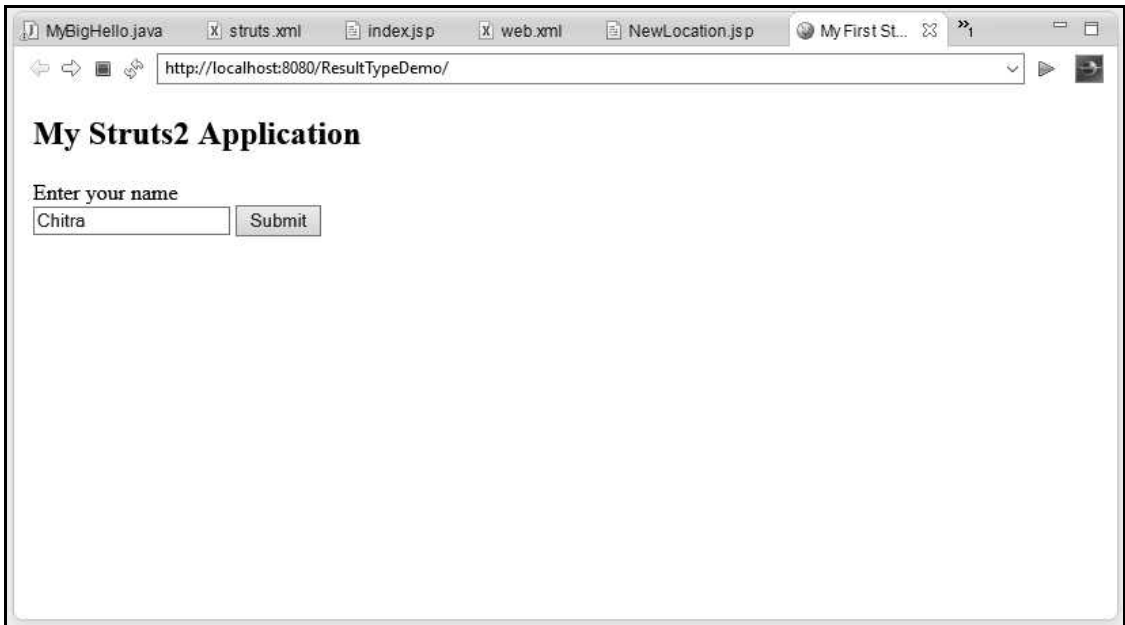
  <package name="default" extends="struts-default">
    <action name = "hello"
      class = "myPkg.MyBigHello"
      method = "execute">
      <result name = "success" type="redirect">
        <param name="location">
          /NewLocation.jsp
        </param>
      </result>
    </action>
    <action name = "index">
      <result >/index.jsp</result>
    </action>
  </package>
</struts>
```

Step 3 : The **NewLocation.jsp** file will be

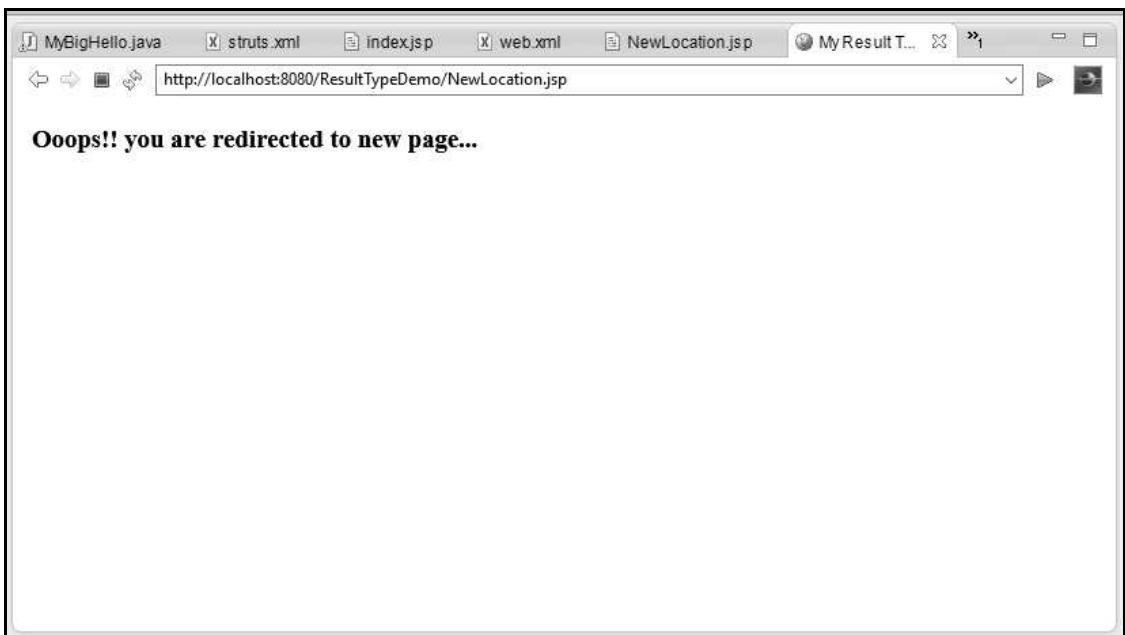
NewLocation.jsp

```
<%@ page language="java" contentType="text/html; charset=ISO-8859-1"
  pageEncoding="ISO-8859-1"%>
<!DOCTYPE html PUBLIC "-//W3C//DTD HTML 4.01 Transitional//EN"
"http://www.w3.org/TR/html4/loose.dtd">
<html>
  <head>
    <title>My Result Type Struts Demo Application</title>
  </head>
  <body>
    <h3>Ooops!! you are redirected to new page...</h3>
  </body>
</html>
```

Step 4 : Now execute the above application and the output can be obtained as follows -



Click the submit button, and you will be redirected to a new page



4.18 Validations

- In struts, validation framework help to perform validation before the action method is executed.
- You can validate data in action class also through **validate()** method.
- In the **validate()** method we can validate the user name, password, age, email, phone number and so on. If the validation fails an error is added using the **addFieldError()** method.

How to use Validations in Struts ?

In this application we will use validation for the name field. The text-box for entering the name is given, if user leave this text box empty then the message entering the valid name is given. This is how the simple validation can be performed in following application.

Step 1 : Start Eclipse. Create a Dynamic Web Project. Copy the required Jar files to **WEB-INF/lib** folder. Now create a Login Page using JSP. This file is created in **WebContent** . In this page one textfields is created to enter valid user name. The code for this JSP script is as follows-

LoginPg.jsp

```
<%@ page language="java" contentType="text/html; charset=ISO-8859-1"
pageEncoding="ISO-8859-1"%>
<%@taglib uri="/struts-tags" prefix="s"%>
<!DOCTYPE html PUBLIC "-//W3C//DTD HTML 4.01 Transitional//EN"
"http://www.w3.org/TR/html4/loose.dtd">
<html>
<head>
<meta http-equiv="Content-Type" content="text/html; charset=ISO-8859-1">
<title>Login Page Demo</title>
<s:head />
</head>
<body>
<s:form action="Login">
<s:textfield name="userName" label="User Name" />
<s:submit value="Submit" />
</s:form>
</body>
</html>
```

Step 2 : Create one more JSP document under **WebContent** folder named **Welcome.jsp** . This JSP file is for displaying welcome message for authentic user. The code is

Welcome.jsp

```
<%@ page language="java" contentType="text/html; charset=ISO-8859-1"
pageEncoding="ISO-8859-1"%>
<%@taglib uri="/struts-tags" prefix="s"%>
<!DOCTYPE html PUBLIC "-//W3C//DTD HTML 4.01 Transitional//EN"
"http://www.w3.org/TR/html4/loose.dtd">
<html>
<head>
<meta http-equiv="Content-Type" content="text/html; charset=ISO-8859-1">
<title>login Page Demo</title>
</head>
<body>
<h1>Welcome, User <s:property value="userName"/> is authentic user!!!</h1>
</body>
</html>
```

Step 3 : Now we will write the Java File for validating the user name. Two validations are used -

- i) the user name must not be blank and ii) the user name must be **root**. For validation purpose we will write one function named **validate()** in this Java program. This method is written in action class, hence this program will be created within package name **myPkg** under the **src** directory.

LoginProg.java

```
package myPkg;
import com.opensymphony.xwork2.ActionSupport;
public class LoginProg extends ActionSupport {
    private String userName;

    public String execute() {
        return SUCCESS;
    }
    public String getUserName() {
        return userName;
    }

    public void setUserName(String userName) {
```

```
        this.userName = userName;
    }
    public void validate() {
        if (getUserName().length() == 0) {
            addFieldError("userName", "UserName.required");
        } else if (!getUserName().equals("root")) {
            addFieldError("userName", "Invalid User");
        }
    }
}
```

Step 4 : Now we will write the configuration files. The struts.xml file under the **WEB-INF/classes** folder will be -

struts.xml

```
<?xml version = "1.0" encoding = "UTF-8"?>
<!DOCTYPE struts PUBLIC
    "-//Apache Software Foundation//DTD Struts Configuration 2.0//EN"
    "http://struts.apache.org/dtds/struts-2.0.dtd">
<struts>
    <constant name="struts.custom.i18n.resources" value="LoginProg" />
    <package name="default" extends="struts-default" namespace="/">
        <action name = "Login"
            class = "myPkg.LoginProg"
            method = "execute">
            <result name = "success">/Welcome.jsp</result>
            <result name = "input">/LoginPg.jsp</result>
        </action>
    </package>
</struts>
```

Step 5 : The web.xml file present inside the WEB-INF folder is modified as -

web.xml

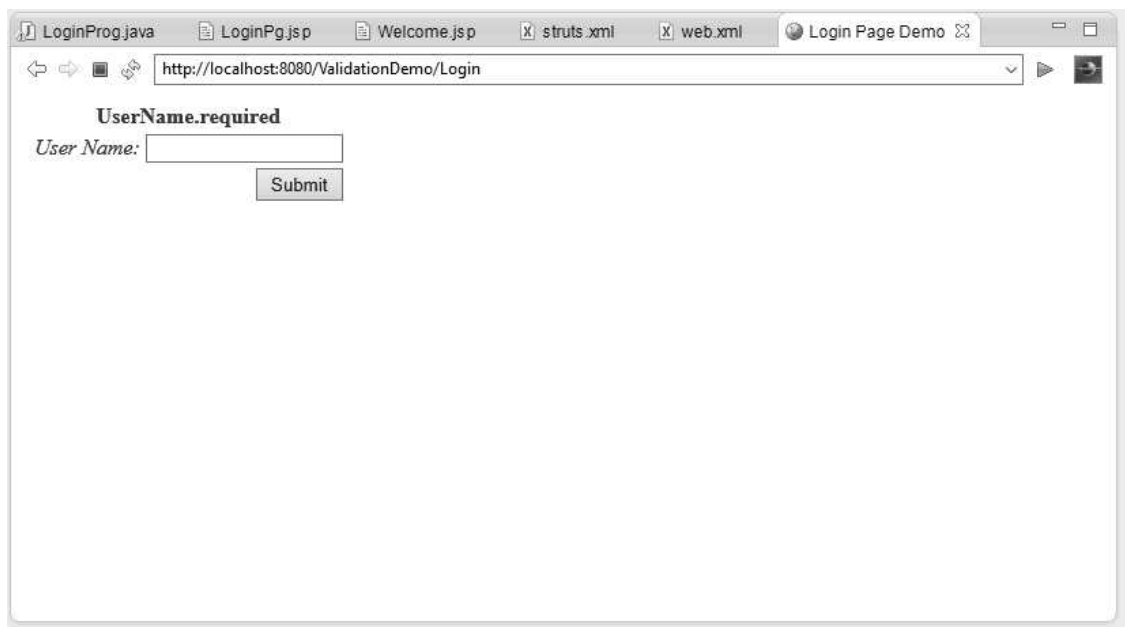
```
<?xml version = "1.0" encoding = "UTF-8"?>
<web-app xmlns:xsi = "http://www.w3.org/2001/XMLSchema-instance"
    xmlns = "http://java.sun.com/xml/ns/javaee"
    xmlns:web = "http://java.sun.com/xml/ns/javaee/web-app_2_5.xsd"
    xsi:schemaLocation = "http://java.sun.com/xml/ns/javaee
    http://java.sun.com/xml/ns/javaee/web-app_3_0.xsd"
    id = "WebApp_ID" version = "3.0">
    <display-name>Struts 2</display-name>
    <welcome-file-list>
```

```
<welcome-file>LoginPg.jsp</welcome-file>
</welcome-file-list>
<filter>
  <filter-name>struts2</filter-name>
  <filter-class>
    org.apache.struts2.dispatcher.ng.filter.StrutsPrepareAndExecuteFilter
  </filter-class>
</filter>
<filter-mapping>
  <filter-name>struts2</filter-name>
  <url-pattern>/*</url-pattern>
</filter-mapping>
</web-app>
```

The above file specifies the entry point for the project. It says that the first page to be displayed is **LoginPg.jsp**.

Step 6 : Now deploy the WAR file to tomcat's webapps directory and run the above application on Apache Tomcat server. The output will be





If we enter the user name as 'root' then we get Welcome message.

4.19 Localization

- Internationalization is a process of translation or localization.
- Internationalization is abbreviated i18n because the word starts with the letter "i" and ends with "n", and there are 18 characters between the first i and the last n.
- Internationalization (i18n) is the process of planning and implementing products and services so that they can easily be adapted to specific local languages and cultures, a process called localization.
- The i18n interceptor provides multi-lingual support for your application.
- It handles setting locale for the action. It can be used if user wants to set his/her locale and get data according to the locale provided.
- A locale is an identifier and represents the local details of the user to the application. The locale can be created by **java.util.Locale** class. When a user session begins, the Locale object is sent in the HTTP request as parameter.

Concept of Resource Bundle

Struts2 makes use of resource bundle to provide multiple languages and locale options to the users. The resource bundle is a resource file in which key-value pairs are stored.

The name to these resource files is given in following naming format

BundleName_language_country.properties

The **language_country** represents the country locale. For example English United States is represented as en_US, For Spanish it will be _es and so on. These properties file can be named as

global.properties : This is a default file for the English language.

global_fr.properties : This file specifies the French locale.

global_cn.properties : It specifies the code in Chinese.

global_hi.properties : It specifies the code in Hindi

How to create application to support Localization ?

Step 1 : Create a Dynamic project. Copy all the necessary .jar files to WEB-INF/lib folder.

Create initial JSP page inside the WebContent folder, name it as **index.jsp**. The code for it will be as follows -

index.jsp

```
<%@ page language = "java" contentType = "text/html; charset = ISO-8859-1"
    pageEncoding = "ISO-8859-1"%>
<%@ taglib prefix = "s" uri = "/struts-tags"%>
<!DOCTYPE html PUBLIC "-//W3C//DTD HTML 4.01 Transitional//EN"
"http://www.w3.org/TR/html4/loose.dtd">
<html>
<head>
<title>Student Form with Multilingual Support</title>
</head>
<body>
<h1><s:text name = "global.heading"/></h1>

<s:url id = "indexEN" namespace="/" action = "locale" >
<s:param name = "request_locale" >en</s:param>
</s:url>

<s:url id = "indexDE" namespace="/" action = "locale" >
<s:param name = "request_locale" >de</s:param>
</s:url>

<s:url id = "indexHI" namespace="/" action = "locale" >
<s:param name = "request_locale" >hi</s:param>
</s:url>

<s:a href="%{indexEN}" >English</s:a>
<s:a href="%{indexDE}" >German</s:a>
<s:a href="%{indexHI}" >Hindi</s:a>
```

```
<s:form action = "studentinfo" method = "post" namespace = "/">
  <s:textfield name = "name" key = "global.name" size = "20" />
  <s:submit name = "submit" key = "global.submit" />
</s:form>
</body>
</html>
```

Step 2 : Now create another JSP page to handle the successful submission of data.

This page will display the message that you are authenticated user. The JSP file is created inside the **WebContent** folder.

The code is very simple and it is

Welcome.jsp

```
<%@ page language = "java" contentType = "text/html; charset = ISO-8859-1"
    pageEncoding = "ISO-8859-1"%>
<%@ taglib prefix = "s" uri = "/struts-tags"%>
<!DOCTYPE html PUBLIC "-//W3C//DTD HTML 4.01 Transitional//EN"
    "http://www.w3.org/TR/html4/loose.dtd">
<html>
<head>
  <title>Success</title>
</head>
<body>
  <s:property value = "getText('global.success')"/>
</body>
</html>
```

In above code have used **getText** method to access the messages. To display text, use a call to **getText** in the property tag.

Step 3 : Now create the Java Class files inside the src folder. First create the package with some suitable name under **Java Resources/src**. Here we will create two Java Class files - first Java class file will take care of Locale and display the **index.jsp** with different languages and second Java class file will handle the form submission in which **execute**, **getXX** and **setXX** methods are defined.

The code for these Java files is as follows

StudLocale.java

```
package myPkg;
import com.opensymphony.xwork2.ActionSupport;
public class StudLocale extends ActionSupport {
    public String execute() {
        return SUCCESS;
    }
}
```

TestAction.java

```
package myPkg;
import com.opensymphony.xwork2.ActionSupport;
public class TestAction extends ActionSupport {
    private String name;
    public String execute() {
        return SUCCESS;
    }
    public String getName() {
        return name;
    }
    public void setName(String name) {
        this.name = name;
    }
}
```

Step 4 : Now we can create **global.properties** files. These files can be created for different languages so that the web page can be displayed in those corresponding languages. Here I will write the global file for displaying the message English only. This file is created inside WEB-INF folder inside which the **classes** folder is created. Save **global.properties** file inside this **classes** folder.

global.properties

```
global.name = Name
global.submit = Submit
global.heading = Select desired Locale
global.success = You are authenticated user!!!
```

In this file, we will display the name of the fields and their corresponding values. For instance for **global.success** the message "You are authenticated user!!!" message must be displayed. For **global.heading** the "Select desired Locale" must be displayed.

Step 5 : Now create the configuration files. First create the **struts.xml** file inside **WEB-INF\classes** folder. The code for this file would be as follows -

struts.xml

```
<?xml version = "1.0" encoding = "UTF-8"?>
<!DOCTYPE struts PUBLIC
    "-//Apache Software Foundation//DTD Struts Configuration 2.0//EN"
    "http://struts.apache.org/dtds/struts-2.0.dtd">
<struts>
    <constant name = "struts.devMode" value = "false" />
    <constant name = "struts.custom.i18n.resources" value = "global" />
    <package name = "localizationdemo" extends = "struts-default" namespace="/">
```

```
<action name = "studentinfo"
  class = "myPkg.TestAction"
  method = "execute">
  <result name = "input">/index.jsp</result>
  <result name = "success">/Welcome.jsp</result>
</action>
<action name = "locale"
  class = "myPkg.StudLocale"
  method = "execute">
  <result name = "success">/index.jsp</result>
</action>
</package>
</struts>
```

Step 6 : Modify the web.xml file present in WEB-INF folder. The code is as follows -

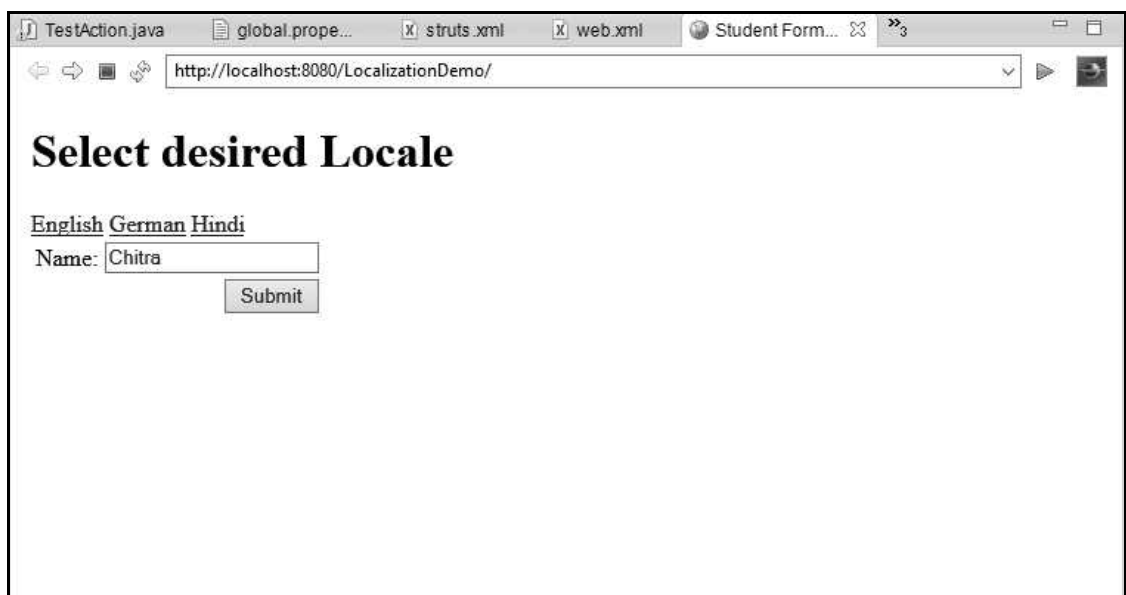
web.xml

```
<?xml version="1.0" encoding="UTF-8"?>
<web-app xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xmlns="http://java.sun.com/xml/ns/javaee"
xmlns:web="http://java.sun.com/xml/ns/javaee/web-app_2_5.xsd"
xsi:schemaLocation="http://java.sun.com/xml/ns/javaee
http://java.sun.com/xml/ns/javaee/web-app_3_0.xsd" id="WebApp_ID" version="3.0">
  <display-name>LocalizationDemo</display-name>
  <welcome-file-list>
    <welcome-file>index.jsp</welcome-file>
  </welcome-file-list>
  <filter>
    <filter-name>struts2</filter-name>
    <filter-class>
      org.apache.struts2.dispatcher.FilterDispatcher
    </filter-class>
  </filter>
  <filter-mapping>
    <filter-name>struts2</filter-name>
    <url-pattern>/*</url-pattern>
  </filter-mapping>
</web-app>
```

Here is the screen-shot for Project Explorer to understand the parent folders of above created files -



Step 7 : Then right click on the project name and click **Export-> WAR** file for exporting the war file in Tomcat directory. Then execute the above application on Tomcat web server. The output will be



Click Submit button and you will get



4.20 Exception Handling

When a program crashes due to some run time error, then this is an undesirable situation. This situation is called exception. Struts provide an efficient way to handle uncaught exception. When such situation occurs, struts redirect to error message display page elegantly.

Following demonstrative application illustrates the exception handling mechanism. In this program we have created error causing statement purposely so that this exception can cause crash of the program. Then this situation is handled using exception handling mechanism.

Step 1 : Create a Dynamic Web project. Copy all the necessary .jar files to WEB-INF/lib folder.

Create initial JSP page inside the **WebContent** folder, name it as index.jsp. The code for it will be as follows -

index.jsp

```
<%@ page language = "java" contentType = "text/html; charset = ISO-8859-1"
    pageEncoding = "ISO-8859-1"%>
<%@ taglib prefix = "s" uri = "/struts-tags"%>
<!DOCTYPE html PUBLIC "-//W3C//DTD HTML 4.01 Transitional//EN"
"http://www.w3.org/TR/html4/loose.dtd">
```

```
<html>
<head>
  <title>My First Struts Demo Application</title>
</head>
<body>
  <h2>My Struts2 Application</h2>
  <form action = "hello">
    <label for = "name">Enter your name</label> <br/>
    <input type = "text" name = "name"/>
    <input type = "submit" value = "Submit"/>
  </form>
</body>
</html>
```

Step 2 : Now create another JSP page to handle the successful submission of data. This page will display the welcome message by user name. The JSP file is created inside the **WebContent** folder. The code is very simple and it is -

MyHello.jsp

```
<%@ page contentType = "text/html; charset = UTF-8" %>
<%@ taglib prefix = "s" uri = "/struts-tags" %>

<html>
<head>
  <title>Exception Demo Application</title>
</head>

<body>
  <h3>Welcome my friend, <s:property value = "name"/></h3>
</body>
</html>
```

Step 3 : Now create the Java Class files inside the **src** folder. First create the package with some suitable name under **Java Resources/src**. This Java class file will handle the form submission in which execute, getXX and setXX methods are defined. Note that purpose, inside the execute method I have created a null string in variable **str**. Then trying to find the Uppercase string of this null string. This action will cause the **NullPointerException** problem. The code for this Java file is as follows -

MyBigHello.java

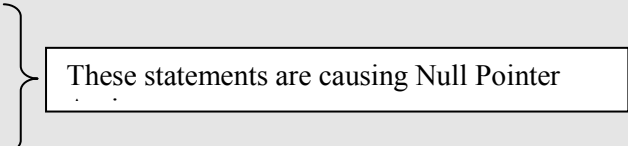
```
package myPkg;
import com.opensymphony.xwork2.ActionSupport;
public class MyBigHello extends ActionSupport{
```

```
private String name;

public String execute() {
    String str=null;
    str=str.toUpperCase();
    return SUCCESS;
}

public String getName() {
    return name;
}

public void setName(String name) {
    this.name = name;
}
}
```



These statements are causing Null Pointer

Step 4 : Now create the configuration files. First create the **struts.xml** file inside **WEB-INF\classes** folder. The code for this file would be as follows -

struts.xml

```
<?xml version = "1.0" encoding = "UTF-8"?>
<!DOCTYPE struts PUBLIC
    "-//Apache Software Foundation//DTD Struts Configuration 2.0//EN"
    "http://struts.apache.org/dtds/struts-2.0.dtd">
<struts>
    <constant name = "struts.devMode" value = "false" />
    <package name="default" extends="struts-default">
        <action name = "hello"
            class = "myPkg.MyBigHello"
            method = "execute">
            <result name = "success">/MyHello.jsp</result>
        </action>
    </package>
</struts>
```

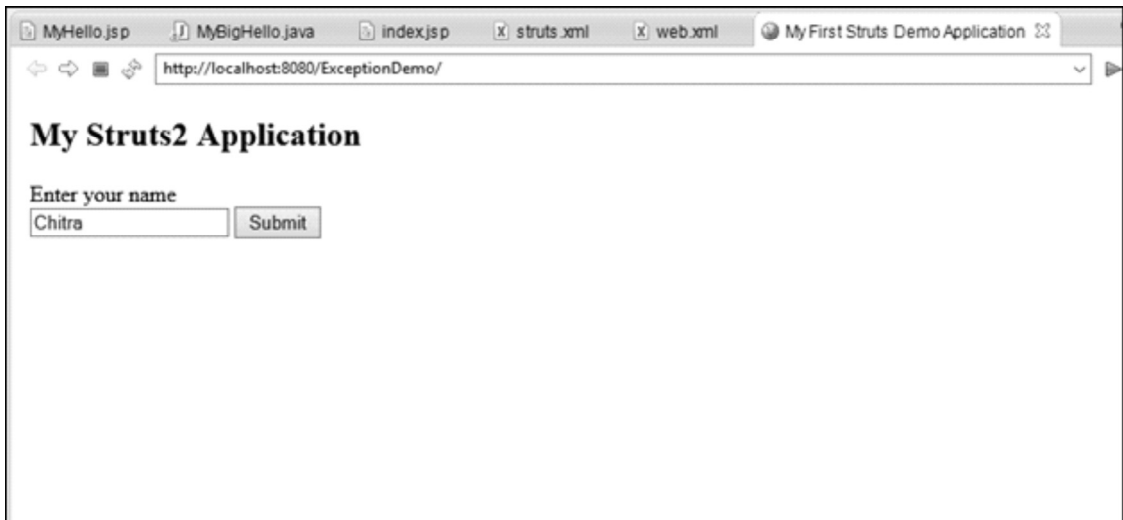
Step 5 : Another configuration file **web.xml** is modified which is present inside the WEB-INF folder.

web.xml

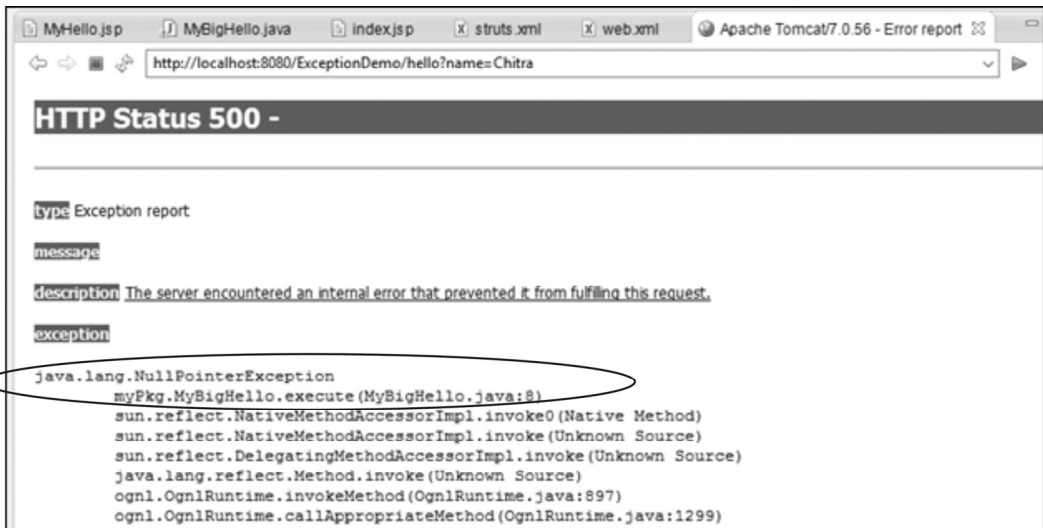
```
<?xml version = "1.0" encoding = "UTF-8"?>
<web-app xmlns:xsi = "http://www.w3.org/2001/XMLSchema-instance"
    xmlns = "http://java.sun.com/xml/ns/javaee"
    xmlns:web = "http://java.sun.com/xml/ns/javaee/web-app_2_5.xsd"
    xsi:schemaLocation = "http://java.sun.com/xml/ns/javaee
    http://java.sun.com/xml/ns/javaee/web-app_3_0.xsd"
    id = "WebApp_ID" version = "3.0">
```

```
<display-name>Struts 2</display-name>
<welcome-file-list>
  <welcome-file>index.jsp</welcome-file>
</welcome-file-list>
<filter>
  <filter-name>struts2</filter-name>
  <filter-class>
    org.apache.struts2.dispatcher.FilterDispatcher
  </filter-class>
</filter>
<filter-mapping>
  <filter-name>struts2</filter-name>
  <url-pattern>/*</url-pattern>
</filter-mapping>
</web-app>
```

Step 6 : Then right click on the project name and click Export-> WAR file for exporting the war file in Tomcat directory. Then execute the above application on Tomcat web server. The output will be



Click Submit button and following exception will be raised -



Step 7 : To overcome this problem we will create a dedicated JSP page to handle exception. I have created a JSP page named **ErrorHandle.jsp** page as follows -

ErrorHandle.jsp

```
<%@ page language = "java" contentType = "text/html; charset = ISO-8859-1"
    pageEncoding = "ISO-8859-1"%>
<%@ taglib prefix = "s" uri = "/struts-tags"%>
<!DOCTYPE html PUBLIC "-//W3C//DTD HTML 4.01 Transitional//EN"
    "http://www.w3.org/TR/html4/loose.dtd">
<html>
<head>
<title></title>
</head>
<body>
<h3>Some Error has occurred in your page!!!</h3>
</body>
</html>
```

Step 8 : Now in order to display the above created JSP page on exception we need to modify our **struts.xml** file. The code will be

struts.xml

```
<?xml version = "1.0" encoding = "UTF-8"?>
<!DOCTYPE struts PUBLIC
    "-//Apache Software Foundation//DTD Struts Configuration 2.0//EN"
    "http://struts.apache.org/dtds/struts-2.0.dtd">
<struts>
```

```
<constant name = "struts.devMode" value = "false" />
<package name="default" extends="struts-default">
  <action name = "hello"
    class = "myPkg.MyBigHello"
    method = "execute">
    <exception-mapping exception = "java.lang.NullPointerException"
      result = "error" />
    <result name = "success">/MyHello.jsp</result>
    <result name = "error">/ErrorHandle.jsp</result>
  </action>
</package>
</struts>
```

In above code, in order to handle the exception the **<exception-mapping>** tag is used. This tag is assigned with the value by the Java exception name **NullPointerException**. Since exceptions in Java are defined in package **java.lang** we denote it as **"java.lang.NullPointerException"**

Step 9 : Now run the above code by exporting WAR file and then on Tomcat server, and following output can be obtained when we enter some user name and click the Submit button.



4.21 Annotations

- Java annotation is a tag used to represent the metadata. The metadata is a data about classes, methods or fields.
- Annotations in Java are used to provide some addition information. For instance - If user is entering his age in the text field, then that age must not be negative or it should not be blank - Such type of information can be provided to the user using annotations.

- A Java annotation in its shortest form looks like this :

```
@Entity
```

- The @ character signals to the compiler that this is an annotation. The name following the @ character is the name of the annotation.
- Struts provides two forms of configuration.
 - The traditional way is to use the **struts.xml** file for all the configurations.
 - The other way of configuring Struts is by using the Java 5 Annotations feature.
- Using annotation we do not need **struts.xml** for configuration. Thus we can achieve zero configuration using annotation.
- Commonly, we can use 3 annotations :
 - 1) **@Action** annotation is used to mark the action class.
 - 2) **@Results** annotation is used to define multiple results for one action.
 - 3) **@Result** annotation is used to display single result.
- To support the annotations we require following files in the Java program to be imported as -

```
import org.apache.struts2.convention.annotation.Action;  
import org.apache.struts2.convention.annotation.Result;  
import org.apache.struts2.convention.annotation.Results;  
import com.opensymphony.xwork2.validator.annotations.*;
```

How to use Annotations in Struts Application ?

Step 1 : Create a project as Dynamic Web Project. Copy following Jar files in the WEB-INF\lib folder.

```
struts2-convention-plugin-x.y.z.jar  
asm-x.y.jar  
antlr-x.y.z.jar  
commons-fileupload-x.y.z.jar  
commons-io-x.y.z.jar  
commons-lang-x.y.jar  
commons-logging-x.y.z.jar  
commons-logging-api-x.y.jar  
freemarker-x.y.z.jar  
javassist-.xy.z.GA  
ognl-x.y.z.jar  
struts2-core-x.y.z.jar  
xwork-core.x.y.z.jar
```

Step 2 : Create the same project as discussed in section 4.18 for validation. The **LoginPg.jsp**, **Welcome.jsp** will be as it is. The only change will be in the Java file. In the Java file we introduce annotations. The code will be as follows -

LoginProg.java

```
package myPkg;
import com.opensymphony.xwork2.ActionSupport;
import org.apache.struts2.convention.annotation.Action;
import org.apache.struts2.convention.annotation.Result;
import org.apache.struts2.convention.annotation.Results;
import com.opensymphony.xwork2.validator.annotations.*;
@Results({
    @Result(name = "success", location = "/Welcome.jsp"),
    @Result(name = "input", location = "/LoginPg.jsp")
})
public class LoginProg extends ActionSupport {
    private String userName;
    @Action(value = "/Login")
    public String execute() {
        return SUCCESS;
    }
    @RequiredFieldValidator( message = "The user Name is required" )
    public String getUserName() {
        return userName;
    }
    public void setUserName(String userName) {
        this.userName = userName;
    }
}
```

Step 3 : Now we just do not require struts.xml file. The web.xml is the only configuration file required. It will be as follow -

web.xml

```
<?xml version = "1.0" encoding = "UTF-8"?>
<web-app xmlns:xsi = "http://www.w3.org/2001/XMLSchema-instance"
    xmlns = "http://java.sun.com/xml/ns/javaee"
    xmlns:web = "http://java.sun.com/xml/ns/javaee/web-app_2_5.xsd"
    xsi:schemaLocation = "http://java.sun.com/xml/ns/javaee
    http://java.sun.com/xml/ns/javaee/web-app_3_0.xsd"
    id = "WebApp_ID" version = "3.0">

    <display-name>Struts 2</display-name>

    <welcome-file-list>
        <welcome-file>LoginPg.jsp</welcome-file>
```

```
</welcome-file-list>
<filter>
  <filter-name>struts2</filter-name>
  <filter-class>
    org.apache.struts2.dispatcher.ng.filter.StrutsPrepareAndExecuteFilter
  </filter-class>
  <init-param>
    <param-name>struts.devMode</param-name>
    <param-value>true</param-value>
  </init-param>
</filter>

<filter-mapping>
  <filter-name>struts2</filter-name>
  <url-pattern>/*</url-pattern>
</filter-mapping>
</web-app>
```

Step 4 : Now Export-War file and execute the above application. If the name field is kept empty and Submit button is clicked then "The user Name is required" message will be displayed.

4.22 Multiple Choice Questions

Q.1 JSP is transformed into _____.

☐ a HTML

☐ b PHP

☐ c Byte Code

☐ d Java Servlet

Q.2 Which programming language does JSP support ?

☐ a JavaScript only

☐ b VB Script only

☐ c Java Only

☐ d All of the above

Q.3 Which of the scripting of JSP not putting content into service method of the converted servlet ?

☐ a Declarations

☐ b Scriptlets

☐ c Expressions

☐ d None of these

Q.4 The difference between Servlets and JSP is the _____.

☐ a translation

☐ b compilation

☐ c syntax

☐ d both a and b

Q.5 Which is not a directive ?

☐ a Include

☐ b Page

☐ c Export

☐ d UseBean

Q.6 Which of the following are the valid scopes in JSP ?

- ☐ a request, page, session, application
- ☐ b request, page, session, global ☐ c response, page, session, application
- ☐ d request, page, context, application

Q.7 “request” is an instance of which one of the following classes :

- ☐ a Request ☐ b HttpRequest
- ☐ c HttpServletRequest ☐ d ServletRequest

Q.8 Which http method send by browser that asks the server to get data ____.

- ☐ a get ☐ b post
- ☐ c option ☐ d put

Q.9 Which attribute specifies a JSP page that should process any exceptions thrown but not caught in the current page ?

- ☐ a ErrorPage attribute ☐ b IsErrorPage attribute
- ☐ c thisErrorPage attribute ☐ d none of these

Q.10 Which technology do we use to mix business logic with presentation logic ?

- ☐ a JSP ☐ b servlet
- ☐ c both a and b ☐ d none of these

Q.11 Which tag is used to execute java source code in JSP ?

- ☐ a declaration tag ☐ b scriptlet tag
- ☐ c expression tag ☐ d both b and c

Q.12 In JSP page directive which attribute defines the MIME(Multipurpose Internet Mail Extension) type of HTTP response ?

- ☐ a import ☐ b info
- ☐ c extends ☐ d ContentType

Q.13 Choose the correct alternative

- ☐ a Servlets are built on JSP semantics and all servlets are compiled to JSP pages for runtime usage.
- ☐ b JSP and servlets are unrelated technologies.
- ☐ c Servlets and JSP are competing technologies for handling web requests. Servlets are being superseded by JSP, which is preferred. The two technologies are not useful in combination.
- ☐ d JSPs are built on servlet semantics and all JSPs are compiled to servlets for runtime usage.

Q.14 What is the initial contact point for handling a web request in a Page-Centric architecture ?

- ☐ a A JSP page
 ☐ b A JavaBean.
☐ c A servlet
 ☐ d A session manager.

Q.15 What is the key difference between using a `<jsp:forward>` and `HttpServletResponse.sendRedirect()` ?

- ☐ a Forward executes on the server while `sendRedirect()` executes on the client.
☐ b Forward executes on the client while `sendRedirect()` executes on the server.
☐ c The two methods perform identically.
☐ d None of these

Q.16 In Struts2 Action classes are the components of ____.

- ☐ a view
 ☐ b controller
☐ c model
 ☐ d deployment

Q.17 In Struts2, What does `validate()` method of `ActionForm` returns ?

- ☐ a ActionErrors
 ☐ b ActionForward
☐ c ActionMapping
 ☐ d ActionError

Q.18 Which of the following methods is overridden by Action class ?

- ☐ a run()
 ☐ b destroy()
☐ c execute()
 ☐ d service()

Answers Keys for Multiple Choice Questions :

Q.1	d	Q.2	c	Q.3	c	Q.4	c	Q.5	c
Q.6	a	Q.7	c	Q.8	a	Q.9	a	Q.10	b
Q.11	b	Q.12	d	Q.13	d	Q.14	a	Q.15	a
Q.16	b	Q.17	b	Q.18	c				

